Concepts Python avancés

Packaging Python et PyPI

Par GALODE Alexandre

Date de publication : 21 septembre 2015

CONFIRMÉ

Il vous est sûrement arrivé de vous dire que si tous les modules/packages étaient stockés de manière centralisée, alors cela simplifierait grandement vos déploiements. Eh bien la fondation Python a eu la même réflexion, et a commencé à travailler pour vous.

Commentez

I - Introduction....4
II - Principe....4
III - L'outil Pip....4
III-A - Le format requirement....5
III-B - Commandes et options....5
III-C - Lister les paquets installés dans un fichier....7
III-D - Installer un groupe de paquets grâce à un fichier requirement....7
III-E - Installer un paquet depuis un fichier....7
III-F - Installer des paquets depuis un dossier....7
III-G - Installer des paquets depuis Mercurial....7
IV - Wheel : le nouveau format officiel....8
V - Packaging Python....8
V-A - Prérequis....8
V-B - Le fichier setup.py....8
V-B-1 - Spécifier le nom et la version du programme....9
V-B-2 - Informations relatives aux auteurs et mainteneurs....9
V-B-3 - Spécifier des mots clés et catégories pour les recherches....9
V-B-4 - Spécifier les URL relatives au code....10
V-B-5 - Spécifier les packages et modules à empaqueter....10
V-B-6 - Décrire l'objectif du code....11
V-B-7 - Spécifier la licence....11
V-B-8 - Spécifier les plateformes....12
V-B-9 - Spécifier les dépendances....12
V-B-10 - Les fichiers additionnels (images, config…)....13
V-C - Le fichier setup.cfg....13
V-D - Création du wheel....13
V-E - Exemple concret....14
VI - PyPI officiel....18
VI-A - Ajout d'un paquet....18
VI-B - Récupération d'un paquet....22
VI-C - Supprimer un paquet....22
VII - PyPI local....22
VII-A - Prérequis....22
VII-B - Installation/désinstallation....23
VII-C - Lancement/contrôle/arrêt du PyPI local....23
VII-D - Concepts basiques....27
VII-D-1 - Serveur....27
VII-D-2 - Index....28
VII-D-3 - Utilisateurs....28
VII-E - Paramétrage principal....28
VII-E-1 - Se connecter et se déconnecter....29
VII-E-2 - Ajout/modification/suppression d'un utilisateur....30
VII-E-3 - Définir les droits admin pour la gestion des utilisateurs....31
VII-F - Index....31
VII-F-1 - Créer un index vierge....31
VII-F-2 - Créer un index héritant d'un autre index....31
VII-F-3 - Associer un index à un utilisateur....32
VII-F-4 - Supprimer un index....32
VII-F-5 - Afficher la liste des index disponibles....32
VII-F-6 - Connaître l'index courant....33
VII-F-7 - Définir l'index courant....34
VII-G - Ajout d'un paquet....34
VII-H - Ajout de documentation Sphinx....35
VII-I - Mise à jour....36
VII-J - Récupération d'un paquet....36
VII-K - suppression d'un paquet....37
VII-L - Navigation dans devpi....37
VII-L-1 - Page d'accueil....37

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

VII-L-2 - Afficher un index sous forme de tableau..... 38
VII-L-3 - Afficher un index sous forme de listing..... 38
VII-L-4 - Page d'accueil d'un package donné..... 39
VII-L-5 - Afficher l'historique d'un package..... 39
VII-L-6 - Affichage de la documentation..... 40
VII-M - Annexe : Les commandes de référence devpi..... 40
VII-M-1 - getjson..... 40
VII-M-2 - Index..... 41
VII-M-3 - Install..... 41
VII-M-4 - List..... 42
VII-M-5 - Login..... 42
VII-M-6 - Logoff..... 43
VII-M-7 - Push..... 43
VII-M-8 - Quickstart..... 43
VII-M-9 - Remove..... 44
VII-M-10 - Test..... 44
VII-M-11 - Upload..... 45
VII-M-12 - Use..... 45
VII-M-13 - User..... 46
VIII - Conclusion..... 47
IX - Remerciements..... 47

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

I - Introduction

Pour une partie d'entre nous, s'est déjà posée la question du déploiement : comment s'organiser, comment gérer ses versions… Autant de questions sans forcément de réponses évidentes. La fondation Python a pensé à nous en créant **PyPI** (prononcer païpi).

Le but est fort simple : standardiser les procédures et nous fournir de véritables outils et moyens de s'organiser tant pour le stockage que pour le déploiement.

À travers cet article, je vous invite à découvrir PyPI, son outil dédié Pip, et bien entendu comment créer vos paquets au format officiel, les « .whl ».

> *Puisque nous allons aborder le packaging dans ce tutoriel, un petit rappel rapide sur les notions de package et de module.*
>
> *Un package Python n'est concrètement qu'un dossier contenant un fichier « __init__.py », fichier qui est vide la plupart du temps. Ce fichier permet juste à Python de savoir qu'il a affaire à un package Python et non à un simple dossier.*
>
> *Un module, lui, est un simple fichier python, contenant du code simple, une/des classe(s), ou un mélange.*
>
> *Un package peut contenir des sous-packages et/ou des modules. Pour de plus amples informations, je vous renvoie **ICI**.*

II - Principe

PyPI, pour PYthon Package Index, est en quelque sorte un serveur centralisé de paquets Python. Il centralise les différents modules/paquets mis à disposition par la communauté et en gère les différentes versions.

Si vous ne disposez pas toujours d'une connexion Internet, vous avez également la possibilité de créer une copie en local.

Mieux encore, si pour diverses raisons, vous ne désirez pas utiliser le PyPI officiel, vous pouvez vous créer un PyPI local totalement ou partiellement vierge et l'utiliser de manière classique.

PyPI ne vient cependant pas seul, puisqu'un outil dédié existe : Pip. Cet outil va permettre d'établir une connexion entre un ordinateur et le serveur PyPI afin d'installer/mettre à jour/désinstaller un module/package avec plus ou moins d'options que nous verrons ensemble.

L'outil Pip va vous permettre d'installer des modules/paquets depuis le serveur PyPI officiel, en ligne, depuis un serveur PyPI local, ou encore depuis un dossier spécifique que vous stipulerez.

III - L'outil Pip

Pip est un outil en ligne de commandes, disponible aussi bien pour la branche 2 que pour la branche 3 de Python, sous Linux, Windows et Mac.

Il est normalement inclus par défaut à partir de Python 2.7.9 (pour la branche 2) et Python 3.4 (pour la branche 3).

Pour toutes les autres versions, il vous faudra l'installer via le script **get-pip**. Une fois ce dernier téléchargé, il vous suffira de lancer la commande :

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

```
python get-pip.py
```

Dans certains cas, et malgré que votre version de Python soit éligible à disposer nativement de Pip, ce dernier peut nécessiter des manipulations manuelles d'installation.

Il existe une version de Pip par version de Python. Ainsi, si vous avez sur votre poste un Python 2.7.9 et un Python 3.4, il existera deux Pip sur votre système. La distinction se fait alors de la manière suivante : Pip<version_en_deux_digits> …

```
1. pip2.7 install 2mp3
2. pip3.4 install 2mp3
```

Dans d'autres cas, la distinction se fera exclusivement de la façon suivante :

```
1. pip install 2mp3
2. pip3 install 2mp3
```

III-A - Le format requirement

Le format requirement est un format très simple permettant de pouvoir stipuler, outre le nom d'un paquet, la version attendue. Les formes possibles d'écriture sont les suivantes:

```
NOM
NOM==version
NOM>=version
NOM<=version
NOM>version
NOM<version
```

Vous avez la possibilité de mettre un ensemble de paquets dans un fichier dit de requirement. Dans ce cas, chaque ligne ne contiendra le nom que d'un seul paquet, lequel devra être saisi au format requirement. Ce fichier porte alors en général le nom de « requirements.txt ».

```
...
Pillow>=2.7.0
piston-mini-client<=0.7.5
psutil==2.1.1
pycrypto==2.6.1
pycups==1.9.72
pycurl==7.19.5
pygobject==3.14.0
...
```

III-B - Commandes et options

Comme tout outil en ligne de commande, Pip possède des commandes et des options. Nous allons voir ici les essentielles.

Avant toute chose, rappelons que Pip fonctionne de la façon suivante

```
pip <commande> [option] paquet
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Commandes	
install	Permet d'installer un paquet
uninstall	Permet de désinstaller un paquet
freeze	Permet d'afficher, ou de créer un fichier contenant l'ensemble des paquets installés, au format requirement
list	Permet de lister les paquets installés sur la machine
show	Permet d'afficher des informations à propos d'un paquet donné
search	Permet de rechercher un paquet, selon un mot clé, sur PyPI (local ou non)
wheel	Crée un fichier Wheel à partir du fichier requirement indiqué (nécessite parfois un paquet supplémentaire)
help	Permet d'afficher l'aide

Options	
-h / --help	Permet d'afficher l'aide
-v / --verbose	Plus verbeux
-V / --version	Permet d'obtenir la version de Pip
-q / --quiet	Moins verbeux
--log-file <path>	Redirige la sortie écran vers un fichier de log. Actif par défaut dans le chemin communiqué par l'aide
--log <path>	Inactif par défaut
--proxy <proxy>	Permet de stipuler un proxy à utiliser, sous la forme [utilisateur:mot_de_passe@]proxy.server:port
--timeout <sec>	Définit le timeout pour le socket (par défaut à 15s)
--exists-action <action>	Action par défaut quand un chemin existe déjà : (s)witch (i)gnore (w)ipe (b)ackup
--cert <path>	Permet de définir le chemin vers des certificats alternatifs
-r / --requirement	Permet d'installer/désinstaller l'ensemble des paquets listés dans un fichier requirement
-y / --yes	Permet de ne pas avoir à confirmer une désinstallation

Plus d'options sont disponibles via un « man pip ».

Ainsi, si nous désirons savoir quels paquets sont installés sur notre machine, nous taperons:

```
pip list
```

list et freeze produiront approximativement la même sortie, mais en utilisant un format d'affichage différent.

Si nous désirons chercher les paquets ayant un lien avec les mp3, nous taperons:

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

```
pip search mp3
```

Lors de l'utilisation de l'outil Pip, vous pouvez parfois voir une ligne vous stipulant qu'une nouvelle version est à jour. N'hésitez alors pas à installer cette nouvelle version afin de bénéficier d'une meilleure stabilité et de potentielles nouvelles options.

III-C - Lister les paquets installés dans un fichier

Comme vu dans le récapitulatif des commandes et options, vous avez la possibilité de lister l'ensemble des paquets installés. Cependant, il faut savoir que vous pouvez générer un fichier grâce à cela :

```
pip freeze > ./requirements.txt
```

Vous disposez alors d'un fichier dit de requirement, souvent nommé requirements.txt. Ce fichier vous permettra ensuite de recréer aisément un environnement similaire.

Libre à vous de supprimer des lignes dans le fichier si vous estimez qu'un des paquets n'a rien à faire dans la liste. De même vous pouvez modifier la version attendue.

III-D - Installer un groupe de paquets grâce à un fichier requirement

Une fois que vous disposez d'un fichier requirement, le déploiement s'avère très facile :

```
pip install -r requirements.txt
```

Pip va alors installer l'ensemble des paquets attendus. Vous serez averti de potentielles erreurs sur votre terminal. Cas échéant, si de nombreux paquets, je vous invite à afficher la liste des paquets installés, à la fin de l'installation, afin de vous assurer que tout a bien été installé.

III-E - Installer un paquet depuis un fichier

Pip permet d'installer des paquets disponibles sur Internet, tel que nous venons de le voir, mais aussi des fichiers locaux. Comment ? Tout simplement comme suit :

```
pip install ./a_installer/mon_paquet.whl
```

III-F - Installer des paquets depuis un dossier

Pouvoir installer une archive est fort pratique, mais si vous en avez 25 à installer ? Bien entendu, vous pourriez saisir 25 fois la même ligne. Mais là aussi Pip à la solution :

```
pip install -f ./mon_dossier
```

Cela installera tous les paquets qui seront trouvés dans « mon_dossier ».

III-G - Installer des paquets depuis Mercurial

Cerise sur le gâteau, Pip sait gérer **les gestionnaires de versions** Mercurial, Git, Svn, et Bazaar .

Nous allons voir ici Mercurial. Dans le cas de ce dernier, Pip supporte les chemins en http, https, static -http et ssh.

```
[-e] hg+http://hg.mon_projet.org/MonProjet
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

```
[-e] hg+https://hg.mon_projet.org/MonProjet
[-e] hg+ssh://hg.mon_projet.org/MonProjet
```

Outre le fait de pouvoir récupérer la dernière révision en date, vous pouvez également en choisir une en particulier, grâce au symbole « @ », en stipulant un hash de révision, un numéro de révision, le nom d'un drapeau, ou bien encore le nom d'une branche précise :

```
[-e] hg+http://hg.mon_projet.org/MonProjet@da39a3ee5e6b
[-e] hg+http://hg.mon_projet.org/MonProjet@2019
[-e] hg+http://hg.mon_projet.org/MonProjet@v1.0
[-e] hg+http://hg.mon_projet.org/MonProjet@special_feature
```

IV - Wheel : le nouveau format officiel

Après avoir autorisé différents formats (tarball, egg, zip…) la fondation Python, via la **PEP427**, définit un format officiel : le wheel.

Une grande vague de migration est **actuellement en cours**, afin de migrer les principaux packages vers ce format.

Au cours de cet article, l'ensemble de nos exemples et explications tourneront donc exclusivement autour de ce format, dont les fichiers portent l'extension « .whl ».

V - Packaging Python

Voici venir un des moments importants dans la vie de tout développeur Python : construire son propre package.

Nous allons voir ici plus précisément les différents fichiers nécessaires, ainsi que leur contenu, et comment s'en servir pour construire votre paquet.

V-A - Prérequis

Afin de pouvoir packager correctement un paquet Python, vous aurez préalablement besoin d'installer les packages setuptools ainsi que wheel.

```
pip install setuptools
pip install wheel
```

V-B - Le fichier setup.py

Il existe différents outils permettant de construire un paquet à partir d'un fichier setup.py. L'outil officiel est distutils, actuellement en cours de remplacement.

L'outil le plus utilisé reste cependant **setuptools**(**doc officielle ici**). C'est cet outil que nous allons utiliser ici.

Le fichier setup.py est le fichier indispensable pour toute distribution de paquets en Python. En effet, c'est lui qui va donner à Python les instructions indispensables à la construction, tels le nom, la version, le ou les auteur(s)…

Le fichier setup.py peut contenir du code, mais en général l'appel à la fonction « setup » suffit.

Voyons maintenant les principaux **paramètres** que vous pouvez communiquer à la fonction setup.


Pour tout paramètre prenant une simple chaîne de caractères, vous êtes limité à 200 caractères.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

V-B-1 - Spécifier le nom et la version du programme

Les premiers éléments, indispensables, à renseigner sont le nom du package à générer, ainsi que sa version.

Ces deux paramètres sont de simples chaînes de caractères.

```
1. from setuptools import setup
2. setup(name = 'nom de mon package',
3.       version = '0.0.1'
4.      )
```

Notez que vous pouvez aussi importer la version et/ou le nom depuis votre package, si vous avez défini la variable __version__, en l'important dans votre fichier setup.py

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__
5.      )
```

V-B-2 - Informations relatives aux auteurs et mainteneurs

Ici, vous allez pouvoir déclarer qui sont les auteurs et les mainteneurs, ainsi que leurs mails respectifs. Là aussi, il s'agit de simples chaînes de caractères

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       author = 'Developpez Dotcom',
6.       author_email = 'developpez@dotcom.fr',
7.       maintainer = 'Developpez Dotcom',
8.       maintainer_email = ' developpez@dotcom.fr'
9.      )
```

V-B-3 - Spécifier des mots clés et catégories pour les recherches

Si jamais vous chargez votre package sur PyPI, par la suite, il sera intéressant de pouvoir effectuer aisément une recherche par mot clé ou par catégorie.

Pour cela, il existe deux paramètres : classifiers et keywords.

Le premier est une liste de strings. Chaque string correspondra à un type prédéfini, dont la liste complète est disponible **ICI**.

Keywords est une simple chaîne de caractères dont les mots clés sont séparés par un espace.

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       keywords = 'exemple documentation setup.py'
6.       classifiers = ['Topic :: Education',
7.                      'Topic :: Documentation']
8.      )
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

V-B-4 - Spécifier les URL relatives au code

Côté URL, vous pouvez spécifier une URL où trouver des informations relatives au code (page du projet, documentation, article de journal, page PyPI…) ainsi qu'une adresse pointant directement vers un lien permettant de charger votre package. Dans les deux cas, il s'agira de simples chaînes de caractères.

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       url = 'http://mon_projet.org',
6.       download_url = 'http://packages.python.org/mon_projet'
7.       )
```

V-B-5 - Spécifier les packages et modules à empaqueter

Indispensable, il faut communiquer à la fonction setup les packages et/ou modules à empaqueter.

Le paramètre de base pour cela est « packages ». Il s'agit d'une liste de chaînes de caractères.

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       packages = ['mon_package']
6.      )
```

Cependant, admettons que pour une raison donnée, votre package à empaqueter ne se situe pas directement au même niveau que votre setup.py. Comment faire ? Eh bien là existe un autre paramètre : « package_dir » qui est un dictionnaire, où la clé est le nom du package, et la valeur le chemin du dossier où trouver le package (relatif à l'emplacement du setup.py).

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       packages_dir = {'mon_package' : 'les_packages_maison'}
6.      )
```

Ici, Python s'attendra à trouver « ./les_packages_maison/mon_package/__init__.py ». Il tombera en erreur sinon.

Cette technique permet de stipuler au cas par cas où trouver les packages. Mais admettons maintenant qu'ils se trouvent tous au même endroit. Dans ce cas, il existe encore une autre façon de faire.

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       packages = ['mon_package'],
6.       packages_dir = {'' : 'les_packages_maison'}
7.      )
```

Dans ce dernier exemple, Python ira toujours chercher les packages stipulés avec le paramètre « packages » dans « ./les_packages_maison ».

Il y a également plus simple que tout cela, sous réserve que vos packages soient placés à un niveau inférieur de votre setup.py.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

En effet, le package setuptools ne vient pas juste avec setup. Vous y trouverez également des fonctions telles que « find_packages() ».

Cette fonction vous permettra de détecter automatiquement, de façon récursive, tout package depuis votre position courante, avec la possibilité d'exclure certains packages.

```
1. from setuptools import setup, find_packages
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       packages = find_packages(exclude=['*.tests.*'])
6.       packages_dir = {'' : 'les_packages_maison'}
7.      )
```

Ici nous demandons à ne pas inclure les niveaux contenant « tests » dans le chemin de modules (ex. : mon_package.tests.demo ne sera pas inclus, mais package.dvp oui).

Maintenant, imaginons que vous n'ayez que des modules et non des packages (ou bien des modules en sus des packages). Le paramètre pour cela est py_modules. Il s'agit d'une liste de chaînes de caractères où l'on précise le nom des modules sans l'extension.

```
1. from setuptools import setup
2. import mon_package
3. setup(name = 'nom de mon package',
4.       version = mon_package.__version__,
5.       packages = ['mon_package'],
6.       py_modules = ['test_demo']
7.      )
```

V-B-6 - Décrire l'objectif du code

Afin de comprendre ce que fait votre code, il est plus aisé d'ajouter des descriptions. Setup.py prend en charge deux types de description : une courte et une longue.

La description courte (en général une phrase) est celle qui est affichée dans PyPI lors d'une recherche. Elle doit décrire brièvement le but du package.

La description longue, elle, peut contenir des informations sur la licence, des exemples, des liens Internet… Pour cette dernière, il est recommandé de disposer d'un fichier texte README, placé au même niveau que setup.py, et qu'on ira lire.

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       description = 'phrase courte expliquant le but de mon projet',
7.       long_description = open(os.path.join(os.path.dirname(__file__), 'README')).read()
8.      )
```

V-B-7 - Spécifier la licence

Bien que vous puissiez spécifier, dans les classifiers, une licence type, à des fins de recherches, il existe également un paramètre permettant d'indiquer par une simple chaîne de caractères la licence utilisée.

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

```
6.       licence = 'GPL V3'
7.      )
```

V-B-8 - Spécifier les plateformes

Il s'agit là d'un paramètre rarement utilisé. Il s'agit d'une simple chaîne de caractères visant à informer des plateformes pouvant utiliser le package.

Les valeurs courantes utilisées sont ALL, WINDOWS, LINUX, MAC.

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       plateformes = 'ALL'
7.      )
```

V-B-9 - Spécifier les dépendances

Vous avez la possibilité de stipuler les dépendances de votre package. Dans setup.py, elles peuvent être de trois types différents :

- Disponible dans PyPI ;
- Non disponible dans PyPI ;
- Autre.

Si le package dont vous dépendez se trouve dans PyPI, il faut utiliser « install_requires ». Les paquets sont alors à stipuler au format requirement, dans une liste de strings :

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       install_requires = ['docutils>=0.3', 'numpy-stl>=1.0.0,<=1.6.2']
7.      )
```

Si les paquets ne sont pas dans PyPI, il faut qu'ils soient disponibles au format tar.gz/egg/whl, en tant que simple fichier .py (en local, réseau, ou sur le net), ou disponibles sur un dépôt (Mercurial (hg+URL), Git (git+URL), ou SVN (svn+URL)).

Il faut alors utiliser « dependency_links ». Il s'agit là aussi d'une liste de strings.

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       dependency_links = ['http://peak.telecommunity.com/snapshots/']
7.      )
```

Enfin, par autre, nous entendons, toutes dépendances qui ne seraient pas totalement obligatoires. Entendez par là, une fonctionnalité spécifique à votre paquet dont tout le monde n'aurait pas besoin (ex. : génération de rapport pdf via reportlab, ou de documentation ReST). Il faut alors utiliser « extras_require ». Cette fois-ci il s'agira d'un dictionnaire. Les clés correspondront à la fonctionnalité, et la valeur sera une liste des paquets attendus.

```
1. from setuptools import setup
2. import os
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

```
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       extras_require = {'PDF': ['ReportLab>=1.2'],
7.                         'ReST': ['docutils>=0.3']}
8.      )
```

V-B-10 - Les fichiers additionnels (images, config…)

Vous avez la possibilité d'empaqueter tout fichier qui serait utile à votre package (par exemple des images) et qui ne serait pas du code informatique. Pour cela, il vous faudra définir une liste de tuples de type (dossier, liste de fichiers). Il vous faudra utiliser le paramètre « data_files ».

```
1. from setuptools import setup
2. import os
3. import mon_package
4. setup(name = 'nom de mon package',
5.       version = mon_package.__version__,
6.       data_files = [('dossier_images',['sous_dossier_image/image1.png'),
7.                     ('dossier_config',['cfg1.cfg', 'cfg2.cfg', 'cfg3.cfg'])]
8.      )
```

V-C - Le fichier setup.cfg

En plus du fichier setup.py, vous pouvez créer un fichier **setup.cfg**. Ce fichier optionnel, qui vise à terme à remplacer le setup.py, contiendra alors des informations complémentaires utiles pour la construction.

Par exemple, stipuler que le code est compatible branche 2 et 3 de Python

```
[bdist_wheel]
universal=1
```

Ou bien encore compatible uniquement avec une version donnée

```
[bdist_wheel]
python-tag = py32
```

Les python tags se fabriquent ainsi : py<version sans le point>-none-<plateforme>. En général, on se contente de « py<version sans le point> ».

V-D - Création du wheel

La création d'un wheel est en soi assez simple. Une fois le setup.py, et potentiellement le setup.cfg, créés, il vous suffira de saisir les lignes suivantes, depuis le dossier où se trouve votre paquet, afin de créer votre wheel :

```
python setup.py test
python setup.py bdist_wheel
```

La première ligne sert à tester le setup.py et vous affichera toute erreur qui pourrait être présente dans ce fichier.

À la suite de la seconde commande, trois nouveaux dossiers sont alors créés :

- build ;
- dist ;
- <nom_projet>.egg-info.

C'est dans « dist » que vous trouverez le paquet généré.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Voilà, vous disposez alors de votre package Python au format standardisé Wheel.

Les exemples présentés ici sont testés sous Linux. Des problèmes semblent survenir sous Windows. En cas de doute, n'hésitez donc pas à lancer une VM ou un dual boot, afin de vous assurer de la source de l'anomalie.

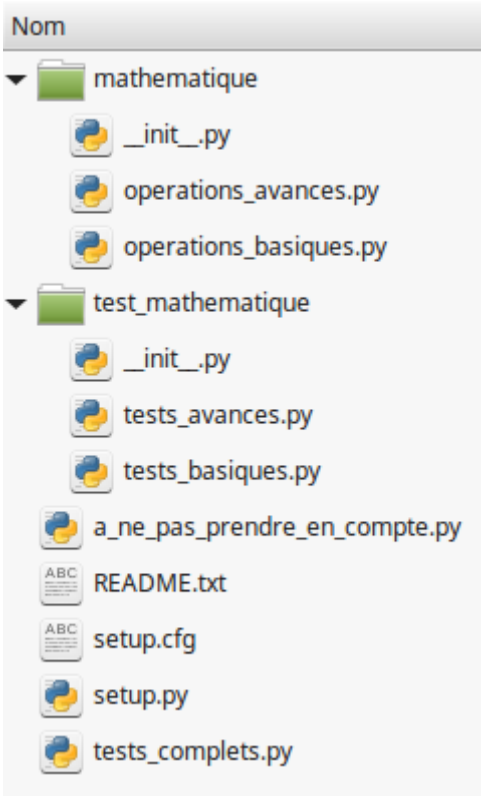
Il semblerait que l'encodage soit parfois mal géré dans certains outils Python. Aussi, si lors du lancement d'une commande, un message lié au caractère ASCII apparaît, commencez par vous assurer que votre chemin ne comporte pas de caractères spéciaux (accents par exemple).

V-E - Exemple concret

Afin de clarifier tout ce que nous venons de voir, nous allons prendre un exemple concret.

Pour cela, imaginons que nous avons créé un package basique nommé « mathematique ». Ce package contiendra 2 modules (opérations_basiques et opérations_avancées). Un second package, « test_mathematique » contiendra des modules destinés à tester le package « mathematique ».

Au même niveau que nos packages, nous aurons également un module Python à ne pas prendre en compte, un module Python permettant de lancer l'intégralité des tests sur notre package « mathematique » et bien entendu, nos fichiers standards nécessaires à la création de notre wheel (setup.py, setup.cfg, README.txt)



Voici maintenant le contenu des différents fichiers (les __init__.py seront vides, exception faite du __init__.py du package « mathematique »).

__init__.py (package mathematique)
```
1. __version__ = '0.0.1'
```

operations_basiques.py
```
1. def addition(a, b):
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

operations_basiques.py

```
    return a + b

def soustraction(a, b):
    return a - b

def multiplication(a, b):
    return a * b

def division(a,b):
    return a / b


if __name__ == '__main__':
    print('addition de 1 & 2:', addition(1, 2))
    print('soustraction de 3 & 2:', soustraction(3, 2))
    print('multiplication de 2 & 3:', multiplication(2, 3))
    print('division de 3 par 4:', division(3, 4))
```

operations_avancees.py

```
def puissance(a, b):
    return a**b

def racine_x(a, b):
    return a**(1./b)


if __name__ == '__main__':
    print('2 puissance 3:', puissance(2,3))
    print('racine cubique de 8:', racine_x(8, 3))
```

tests_basiques.py

```
import sys
import os
sys.path.append(os.path.join(os.path.dirname(os.path.realpath(__file__)), os.pardir))

from mathematique.operations_basiques import addition
from mathematique.operations_basiques import soustraction
from mathematique.operations_basiques import multiplication
from mathematique.operations_basiques import division

def test_addition():
    result = addition(1, 2)
    if result == 3:
        print('Test addition OK')
    else:
        print('Test addition KO')

def test_soustraction():
    result = soustraction(3, 2)
    if result == 1:
        print('Test soustraction OK')
    else:
        print('Test soustraction KO')

def test_multiplication():
    result = multiplication(2, 3)
    if result == 6:
        print('Test multiplication OK')
    else:
        print('Test multiplication KO')

def test_division():
    result = division(float(3), 4)
    if result == 0.75:
        print('Test division OK')
    else:
        print('Test division KO')


```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

tests_basiques.py
```
39. if __name__ == '__main__':
40.     test_addition()
41.     test_soustraction()
42.     test_multiplication()
43.     test_division()
```

tests_avances.py
```
1. import sys
2. import os
3. sys.path.append(os.path.join(os.path.dirname(os.path.realpath(__file__)), os.pardir))
4.
5. from mathematique.operations_avances import puissance
6. from mathematique.operations_avances import racine_x
7.
8. def test_puissance():
9.     result = puissance(2, 3)
10.     if result == 8:
11.         print('Test puissance OK')
12.     else:
13.         print('Test puissance KO')
14.
15. def test_racine_x():
16.     result = racine_x(float(8), 3)
17.     if result == 2:
18.         print('Test racine_x OK')
19.     else:
20.         print('Test racine_x KO')
21.
22.
23. if __name__ == '__main__':
24.     test_puissance()
25.     test_racine_x()
```

tests_complets.py
```
1. from test_mathematique.tests_basiques import test_addition
2. from test_mathematique.tests_basiques import test_soustraction
3. from test_mathematique.tests_basiques import test_multiplication
4. from test_mathematique.tests_basiques import test_division
5. from test_mathematique.tests_avances import test_puissance
6. from test_mathematique.tests_avances import test_racine_x
7.
8. if __name__ == '__main__':
9.     test_addition()
10.     test_soustraction()
11.     test_multiplication()
12.     test_division()
13.     test_puissance()
14.     test_racine_x()
```

README.txt
```
Ceci est le contenu du fichier README.TXT. Il s'agit d'un fichier texte ou RST contenant la description
 longue du paquet. Deporter cette description dans un fichier externe a setup.py permet d'ameliorer la
 lisibilite du code de setup.py.
Ce fichier contient normalement au minimum le nom du paquet, son but, la licence et un hyperlien si
 possible. Il peut etre utile d'ajouter en sus un exemple.
```

a_ne_pas_prendre_en_compte.py
```
1. string1 = 'ceci est un fichier python ne contenant que deux strings'
2. string2 = 'a ne pas prendre en compte a la creation du package'
```

setup.py
```
1. from setuptools import setup, find_packages
2. import os
3. import mathematique
4.
5. setup(name = 'pkg_dvp_mathematique',
6.       version = mathematique.__version__,
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

setup.py

```
7.      author = 'dvp',
8.      author_email = 'dvp@dvp.com',
9.      maintainer = 'dvp',
10.      maintainer_email = 'dvp@dvp.com',
11.      keywords = 'dvp developpez.com deusyss package Python',
12.      classifiers = ['Topic :: Education', 'Topic :: Documentation'],
13.      packages = ['mathematique', 'test_mathematique'],
14.      description = 'demonstration de creation d\'un package Python',
15.      long_description = open(os.path.join(os.path.dirname(__file__), 'README.txt')).read(),
16.      license = 'GPL V3',
17.      platforms = 'ALL',
18.     )
```

```
[bdist_wheel]
universal=1
```

Voilà. Comme vous le constatez, rien de compliqué. Le code se contente juste de mettre à disposition des fonctions, via le package « mathematique », effectuant du calcul relativement simple.

Le package test_« mathematique », lui possède deux modules, chacun affecté à un module du package « mathematique », afin d'en tester les fonctionnalités.

Enfin, au plus haut niveau, un module tests_complets permet de lancer l'intégralité des tests sur le package « mathematique ».

Voilà pour la partie code. À côté, nous retrouvons, un fichier README, deux fichiers setup, et un fichier qu'il ne faudra pas prendre en compte dans notre wheel. Pour ce dernier, il suffit de ne pas y faire référence dans notre setup. Si nous voulions l'embarquer dans le wheel, il suffirait alors de le déclarer en tant que module à prendre en compte.

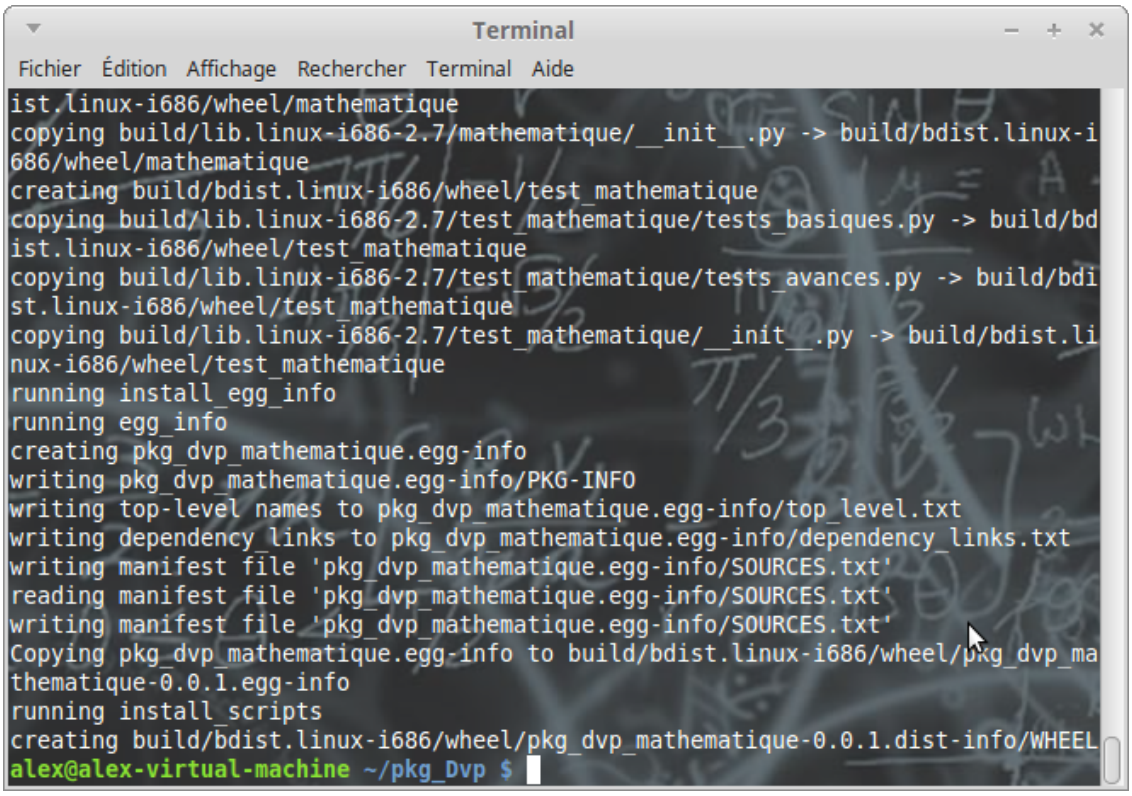
Cet exemple basique, bien que non exhaustif, va vous permettre de mettre en œuvre les concepts vus précédemment.

Maintenant, placez-vous dans le dossier « Dossier_de_packagetage », et lancez la commande :

```
python setup.py bdist_wheel
```

Vous disposez alors des trois dossiers cités précédemment. Le dossier nommé dist contiendra votre wheel.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



Rendez-vous sur une autre machine ou une VM (notez que vous pouvez également le faire sur le même poste que le packetage), rendez-vous dans le dossier contenant le wheel et exécutez la commande :

```
pip install <nom_du_paquet>
```

Et voilà, votre package s'est parfaitement installé. En ouvrant une console Python, vous pouvez désormais faire un « import mathematique ». Puis un help (mathematique) vous communiquera plus d'informations et vous indiquera que le déploiement de votre paquet s'est bien déroulé.

Pour la suite de ce tutoriel, nous nous appuierons sur ce package maison.

VI - PyPI officiel

Maintenant que nous avons vu comment packager nos codes, il est temps de voir les interactions avec PyPI. Commençons avec le PyPI officiel, en ligne.

VI-A - Ajout d'un paquet

Nous partirons ici des postulats suivants :

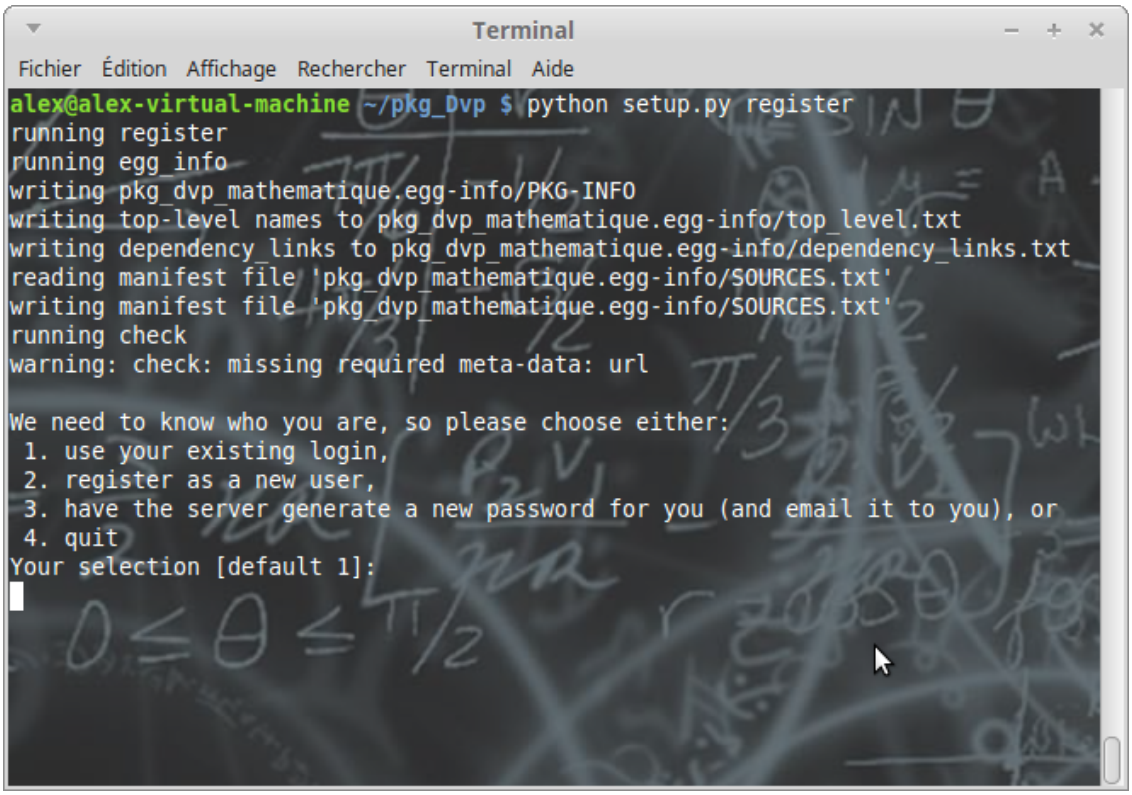
- vous disposez d'un paquet fonctionnel ;
- vous disposez du setup.py qui l'accompagne (avec un numéro de version) ;
- le nom du paquet est disponible (doublons non autorisés sur PyPI) ;
- le paquet wheel est installé ;
- le paquet twine est installé (cf la note plus bas pour l'installation).

Pour démarrer, ouvrez un terminal, depuis le dossier contenant votre setup.py et tapez :

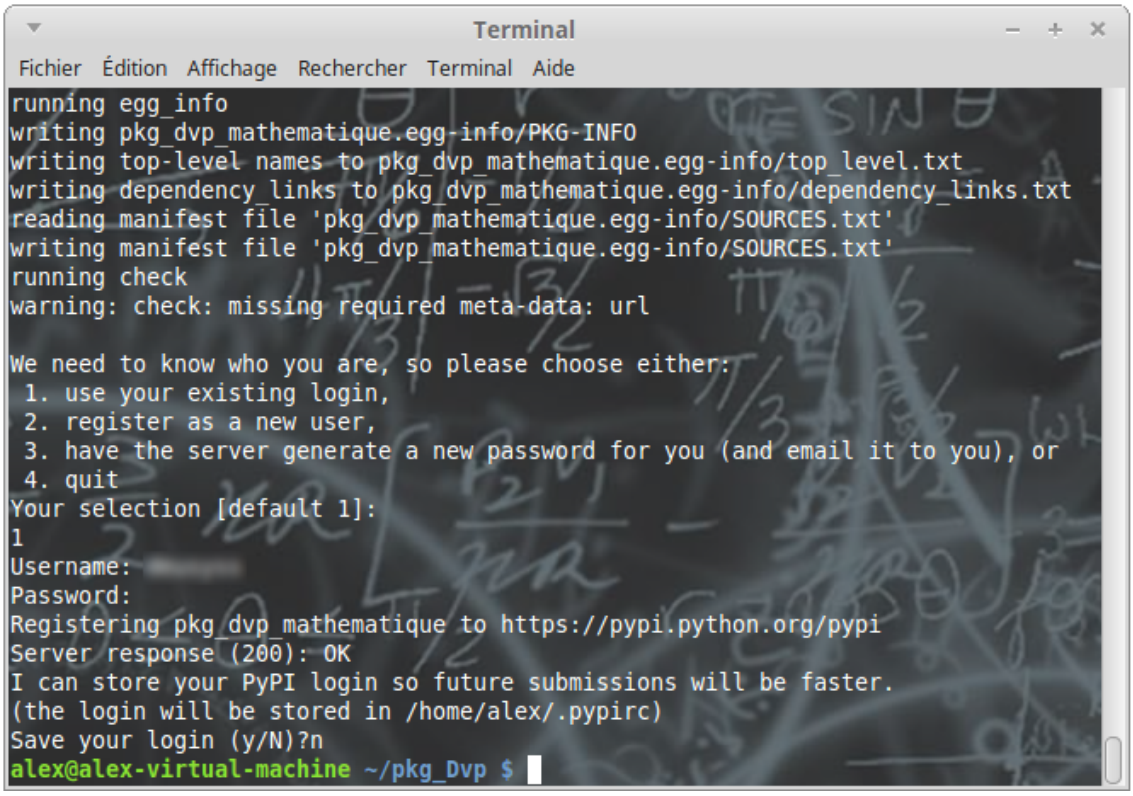
```
python setup.py register
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

Un menu va alors apparaître



S'il s'agit de votre première connexion à PyPI, il vous faudra d'abord créer un compte. La documentation officielle recommande d'utiliser le formulaire en ligne. Cas échéant, vous pourrez utiliser l'option 2. Si vous possédez déjà un compte, utilisez l'option 1.



Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

Si tout se passe bien, vous aurez alors un message indiquant que l'enregistrement du nom de votre paquet est OK (réponse de type 200 OK).

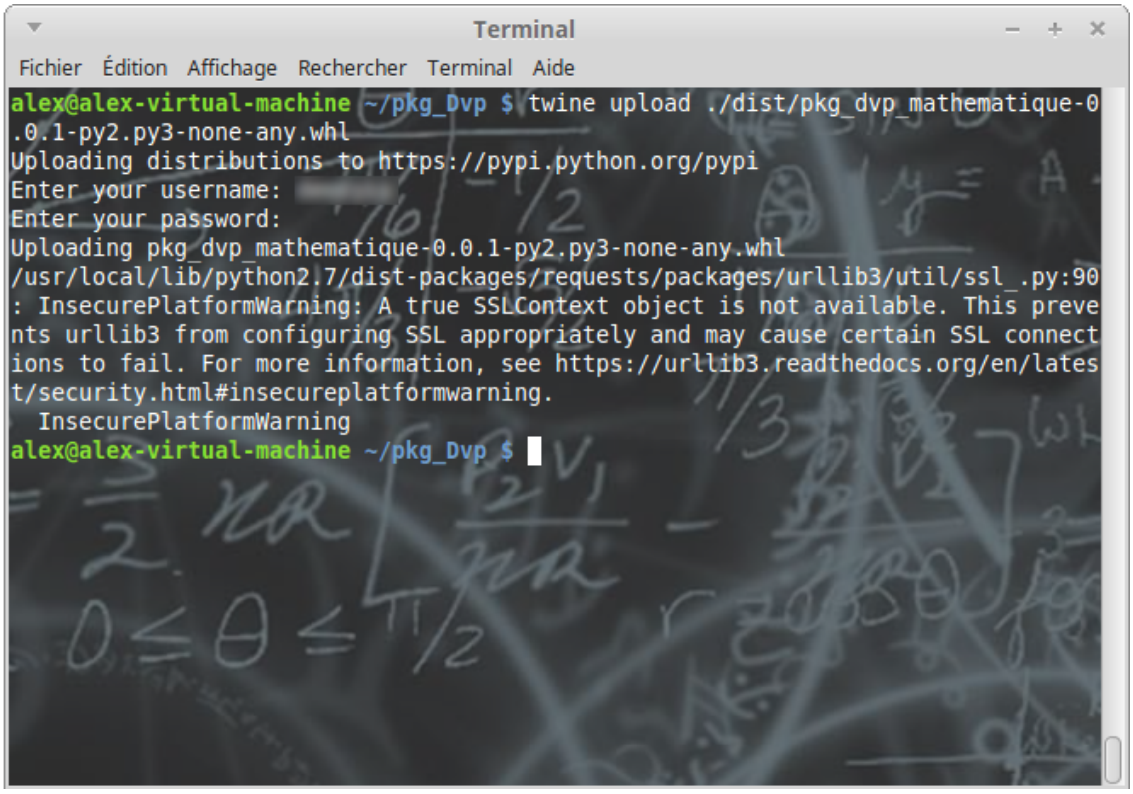
Enfin, il ne vous reste plus qu'à mettre votre paquet en ligne. Pour cela, nous allons utiliser «**twine**».

Pour installer twine, il faut saisir la ligne suivante :

```
pip install twine pyopenssl ndg-httpsclient pyasn1
```

Ce sont là les paquets minimums. Ensuite, selon votre OS, il se peut que vous deviez ajouter quelques packages supplémentaires.

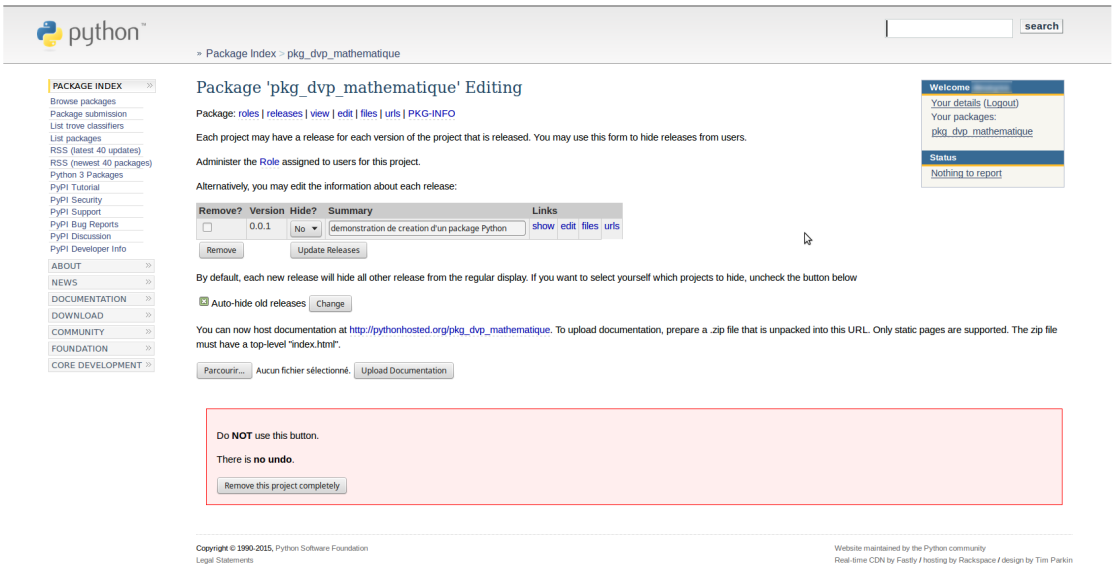
```
twine upload dist/<paquet .whl>
```



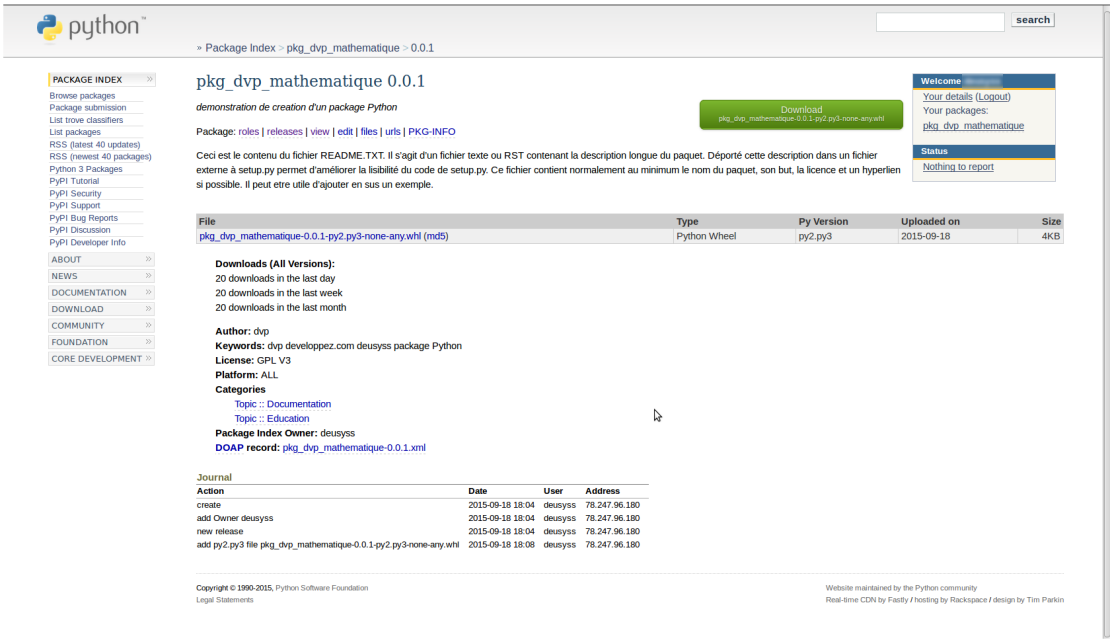
Les plus curieux pourront aller consulter la documentation officielle de la **procédure à suivre**.

Et voilà, notre paquet est en ligne. Si maintenant nous nous connectons, à notre compte PyPI, nous avons accès à la page dédiée de notre package :

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



Comme nous pouvons le voir, beaucoup d'opérations sont alors disponibles en ligne. En cliquant sur view, vous aurez l'aperçu de l'affichage auprès des autres utilisateurs.



On peut voir en bas des informations reprises depuis le setup.py.

Pour tout update, veuillez penser à mettre à jour le numéro de version sans quoi l'opération tombera en échec.

Ces derniers mois, l'outil de recherche du site PyPI présentait des défaillances. Aussi, une fois votre paquet ajouté, il est accessible via l'adresse ***https://PyPI.python.org/PyPI/****<nom_du_paquet>, mais n'apparaît pas forcément lors d'une recherche. De plus l'indexation n'est pas instantanée. Il se peut que vous deviez attendre une semaine avant de voir votre paquet apparaître.*

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

VI-B - Récupération d'un paquet

Pour récupérer un paquet, vous avez trois solutions principales possibles :

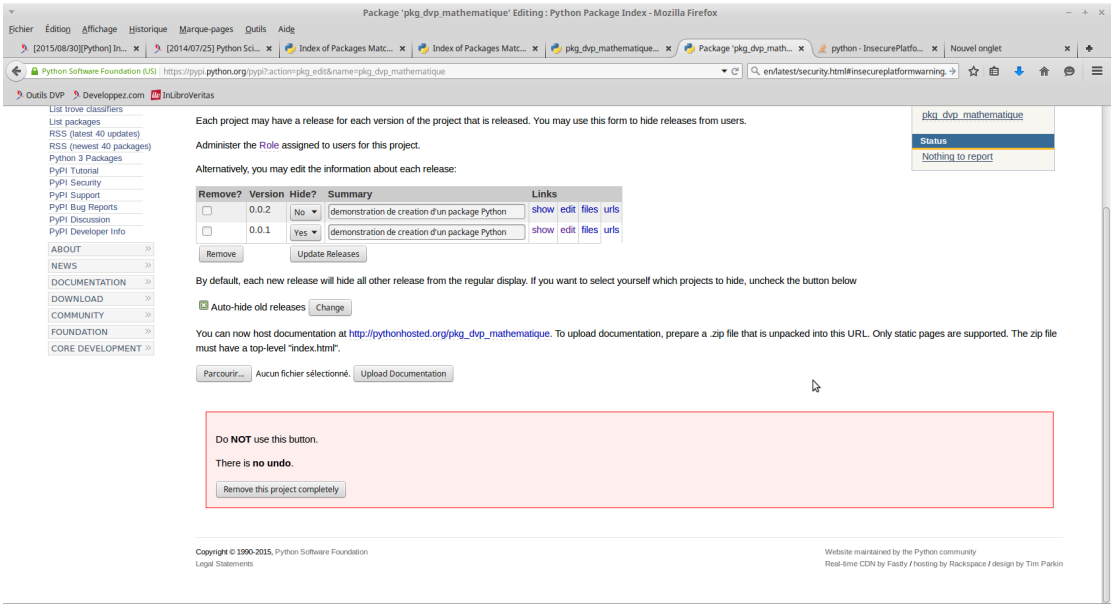
- vous connecter sur la page du paquet, et télécharger le paquet ;
- le télécharger avec un wget sous Linux, en ligne de commande ;
- passer par Pip qui le chargera et l'installera d'office pour vous, si vous le désirez.

VI-C - Supprimer un paquet

La suppression d'un paquet est manuelle, sur le PyPI. Pour cela, rendez-vous sur le PyPI, puis identifiez-vous.

Ouvrez alors la page du paquet concerné, et choisissez le lien « release ».

Des cases à cocher, et des boutons sont alors à votre disposition pour supprimer tout ou partie du package.



VII - PyPI local

Comme nous venons juste de le voir, avec le PyPI officiel, rien de bien compliqué. Passons maintenant au PyPI local.

Afin de le créer, nous nous baserons sur l'outil **devpi**. Il s'agit d'un outil permettant de se créer en local ou sur serveur, un clone du serveur PyPI officiel.

Par la suite, vous disposez également de la possibilité de n'activer votre serveur PyPI qu'en local, ou d'activer une interface réseau afin de la partager sur votre réseau interne, ou sur Internet.

*Si vous ne trouvez pas réponse à votre question dans les sections qui suivent, je vous renvoie vers le **manuel utilisateur**. Une **annexe** est également disponible, en fin de ce tutoriel, afin de résumer les principales commandes DEVPI.*

VII-A - Prérequis

Pour pouvoir installer PyPI en local, rien de plus simple. Vous devez disposer :

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

- d'une connexion Internet ;
- de Python ;
- de Pip.

Une fois que vous avez tout cela, vous pouvez envisager l'installation de PyPI en local.

VII-B - Installation/désinstallation

Pour l'installation rien de plus simple : il est disponible sur PyPI, d'où le besoin d'une connexion Internet.

Il est tout à fait possible d'installer directement devpi en ayant téléchargé préalablement son paquet sur PyPI. Pour cela, suivez la procédure décrite dans la section concernant Pip.

Ensuite, dans un terminal, saisissez

```
pip install devpi
pip install devpi-server
```

Pour le désinstaller, saisissez

```
pip uninstall devpi
pip uninstall devpi-server
```

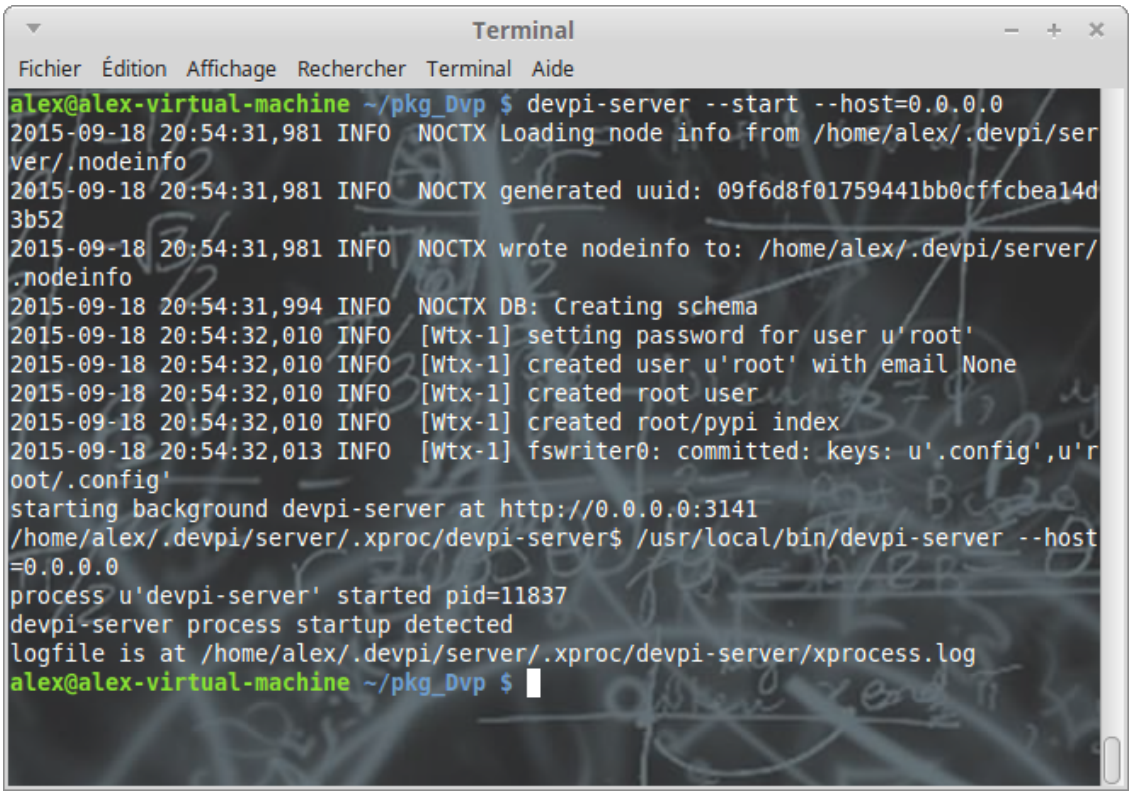
Assurez-vous de bien disposer des droits administrateur sur votre système.

VII-C - Lancement/contrôle/arrêt du PyPI local

Pour démarrer le serveur PyPI local, rien de plus simple :

```
devpi-server --start
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



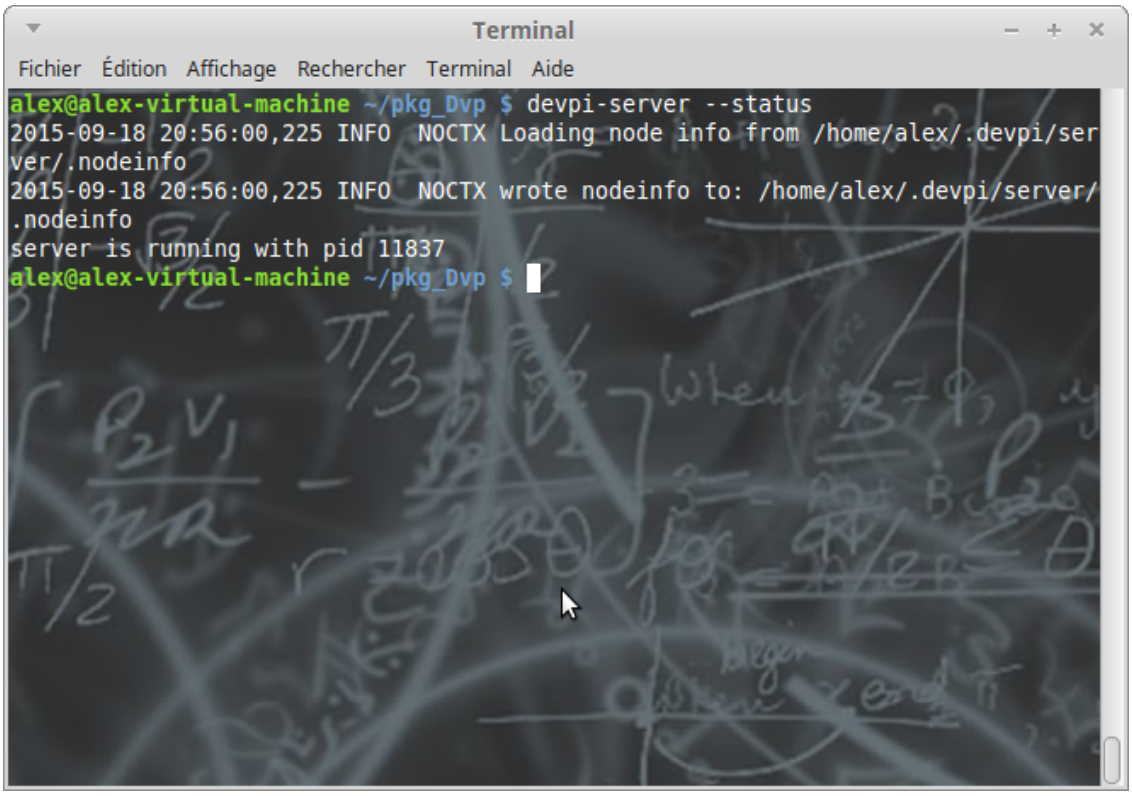
Le serveur est alors disponible sur **http://localhost:3141** par défaut.

3141 est une référence au nombre pi (3.141) afin de faire un jeu de mots avec PyPI et devpi

Par la suite vous pouvez consulter son état :

```
devpi-server --status
```

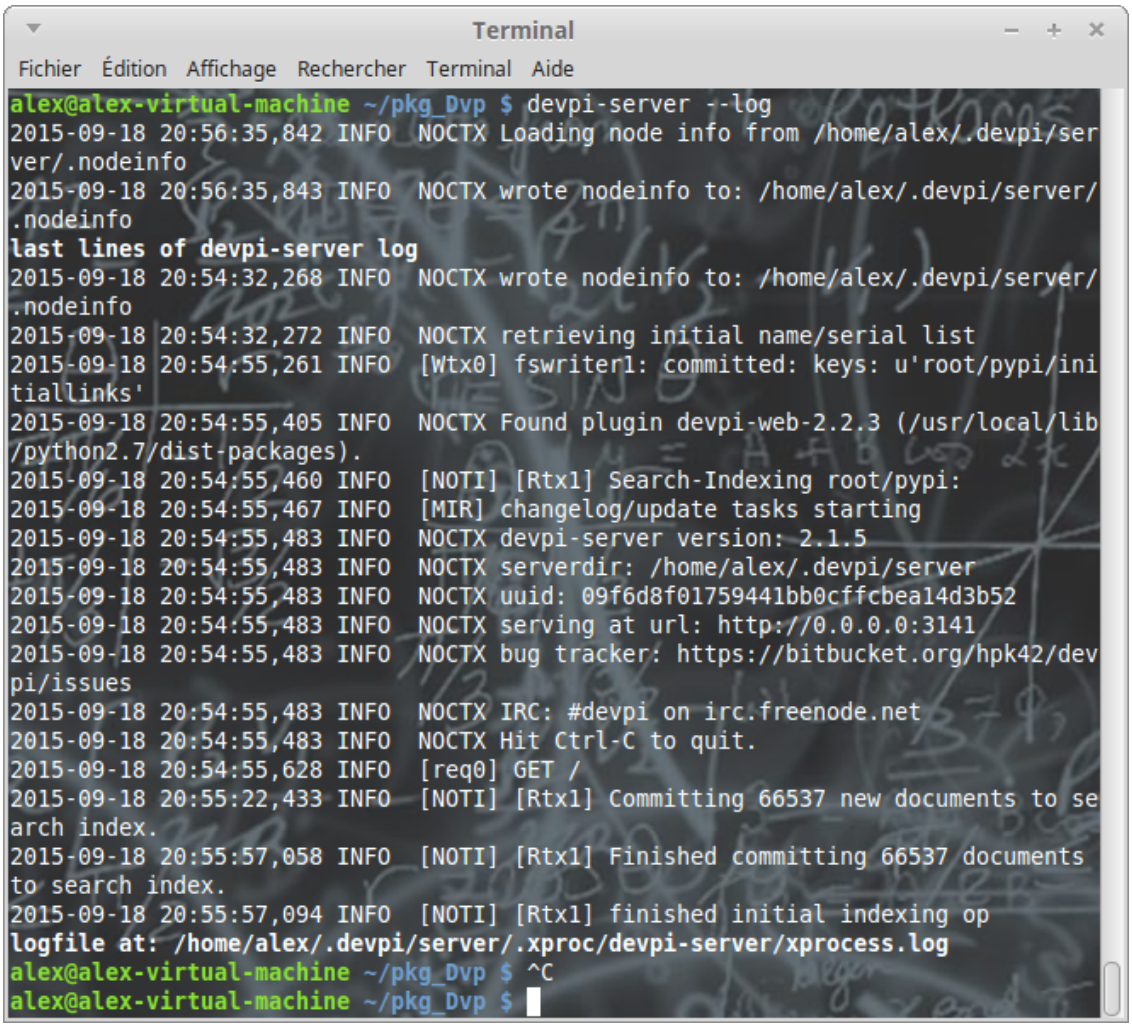

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



Ou bien encore, consulter ses logs :

```
devpi-server --log
```

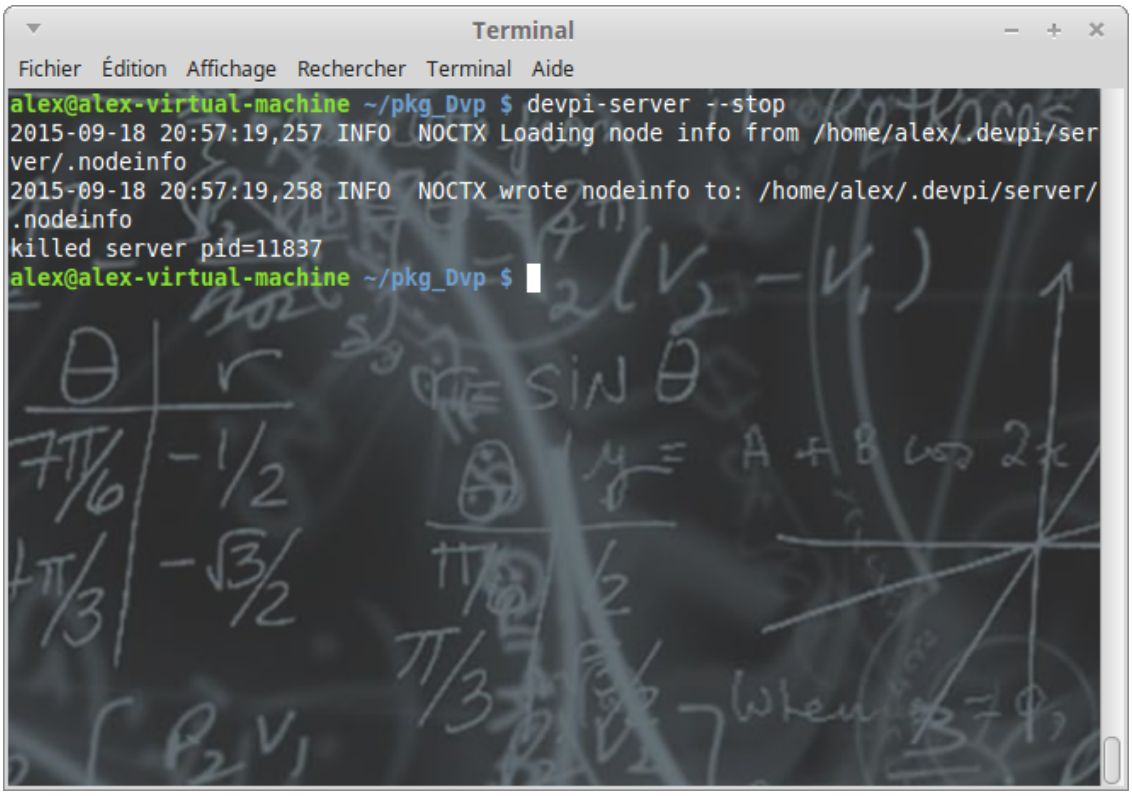

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



Enfin, pour l'arrêter, utilisez la commande qui suit :

```
devpi-server --stop
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



Bien entendu, il ne s'agit pas là des seules options disponibles pour devpi-server. La **documentation officielle** vous informera bien plus. Cependant, voici celles qui pourraient réellement vous être utiles :

Option	Description
--host HOST	Permet de stipuler une adresse autre que localhost pour accéder au PyPI. Pour autoriser un accès autre qu'en local, utilisez « --host=0.0.0.0 »
--port PORT	Permet de stipuler un port autre que 3141
--theme THEME	Permet de stipuler un dossier pour personnaliser l'apparence de votre PyPI local (consultez la **documentation officielle** pour plus d'information)
--version	Permet d'afficher la version en cours d'exécution
--export PATH	Permet d'exporter la BDD de devpi-server dans le chemin indiqué
--import PATH	Permet d'importer une BDD devpi-server depuis le chemin indiqué

VII-D - Concepts basiques

VII-D-1 - Serveur

Le serveur devpi fonctionne à la manière de PyPI, mais n'est pas un clone réel de PyPI.

Ce qu'il faut comprendre, c'est que si le fonctionnement avec Pip est similaire (à l'exception que vous devez stipuler expressément l'adresse du serveur à utiliser), en accès web, l'interface graphique n'a rien à voir.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

De plus, selon le type d'index, le fonctionnement diffère également.

Ainsi, si vous utilisez l'index par défaut /root/PyPI, à la demande d'un paquet, devpi va d'abord regarder si le paquet est disponible en mémoire (sur votre installation personnelle). Si ce n'est pas le cas, alors il cherchera à se connecter au PyPI officiel afin de le mettre en cache sur votre machine. Il n'aura ainsi, par la suite, plus à se connecter sur PyPI pour en disposer.

En revanche, dans le cas d'index personnel, s'il vous faudra charger l'ensemble de vos paquets, ces derniers seront directement hébergés en local.

VII-D-2 - Index

Les index sont des conteneurs vous permettant de stocker paquets, résultats de tests et documentation.

Un index peut être totalement autonome, ou bien dériver lui-même d'un autre index, que cet index de référence appartienne à l'utilisateur courant ou non. Vous pouvez disposer de plusieurs index.

Cependant, cela va plus loin.

Ces conteneurs possèdent deux principaux modes de fonctionnement : volatil et non volatil. Par volatil, il faut comprendre avoir le droit d'écraser les données par d'autres d'une même version. Un index non volatil ne pourra être effacé ou modifié. La principale utilisation des index non volatils se trouve en production.

Par défaut, à la création, l'utilisateur root dispose d'un index PyPI. Il s'agit d'un index spécial unique. En effet, ce dernier est en lecture seule et aucun upload n'est possible vers cet index.

VII-D-3 - Utilisateurs

Les utilisateurs constituent une composante complémentaire aux index.

Prenons l'exemple d'une entreprise. En effet, tous les employés n'ont pas besoin d'avoir accès aux mêmes paquets.

Ainsi, l'intégration nécessitera des paquets spécifiques aux installations sur machines, l'équipe développement, des paquets spécifiques à des mesures physiques…

Trois fonctionnements sont alors possibles :

- tout mettre au même endroit (un utilisateur, un index) (et risquer des conflits de noms en plus du mélange des paquets des différentes équipes) ;
- créer un utilisateur par équipe avec chacun un ou plusieurs index ;
- créer un utilisateur par projet, avec un index ou plusieurs par équipe.

Force est de constater que les deux dernières solutions sont les plus propres. Le compartimentage affiné est rendu possible par l'utilisation conjointe utilisateur/index. En effet, ne disposer que de la possibilité de créer des utilisateurs ou que des index reviendrait à retomber sur la solution 1, avec les désagréments que cela pourrait engendrer.

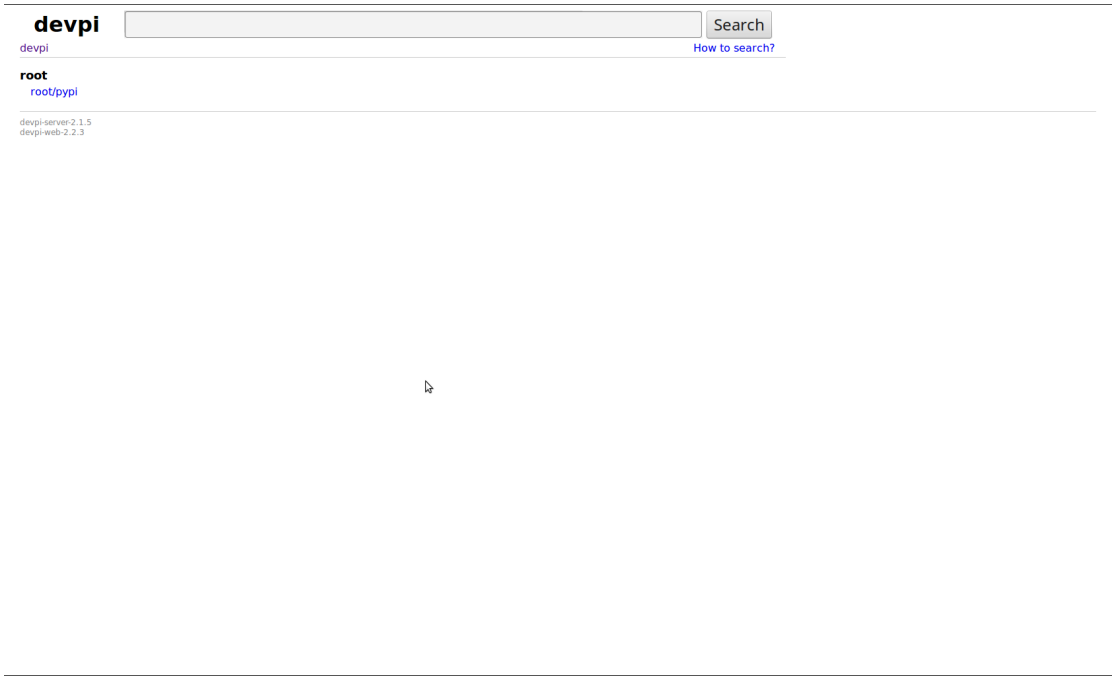
De plus, afin de sécuriser un peu plus l'accès aux paquets, les utilisateurs disposent (optionnellement) de mots de passe.

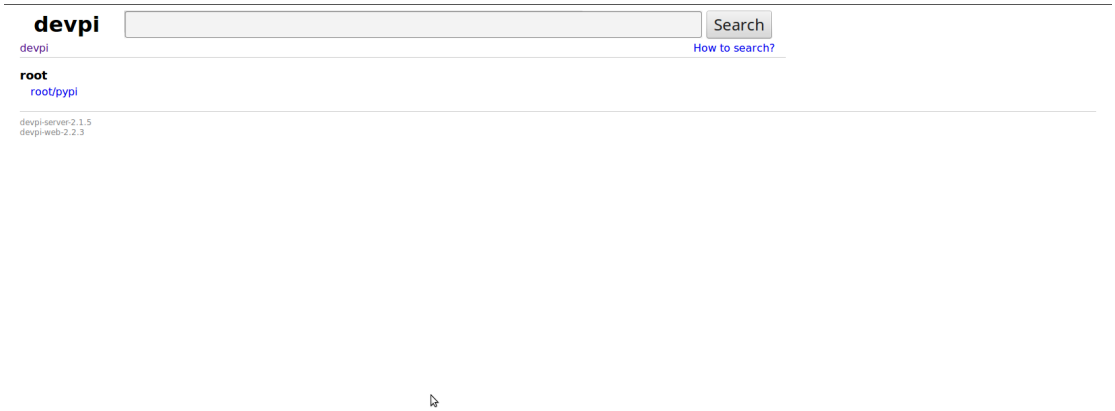
VII-E - Paramétrage principal

Pour effectuer du paramétrage, il vous faudra avant tout définir auprès de devpi, la localisation du serveur, puis indiquer l'index à utiliser :

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

```
devpi use http://localhost:3141
devpi use /root/PyPI
```





VII-E-1 - Se connecter et se déconnecter

Au cours des diverses manipulations, il vous faudra vous authentifier en tant que root, ou en tant qu'utilisateur. Attention toutefois, car à la création, root ne possède pas de mot de passe :

```
devpi login root --password ''
devpi logoff
```

Pour la suite des opérations, assurez-vous bien d'être systématiquement identifié. Au minimum en tant que root

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

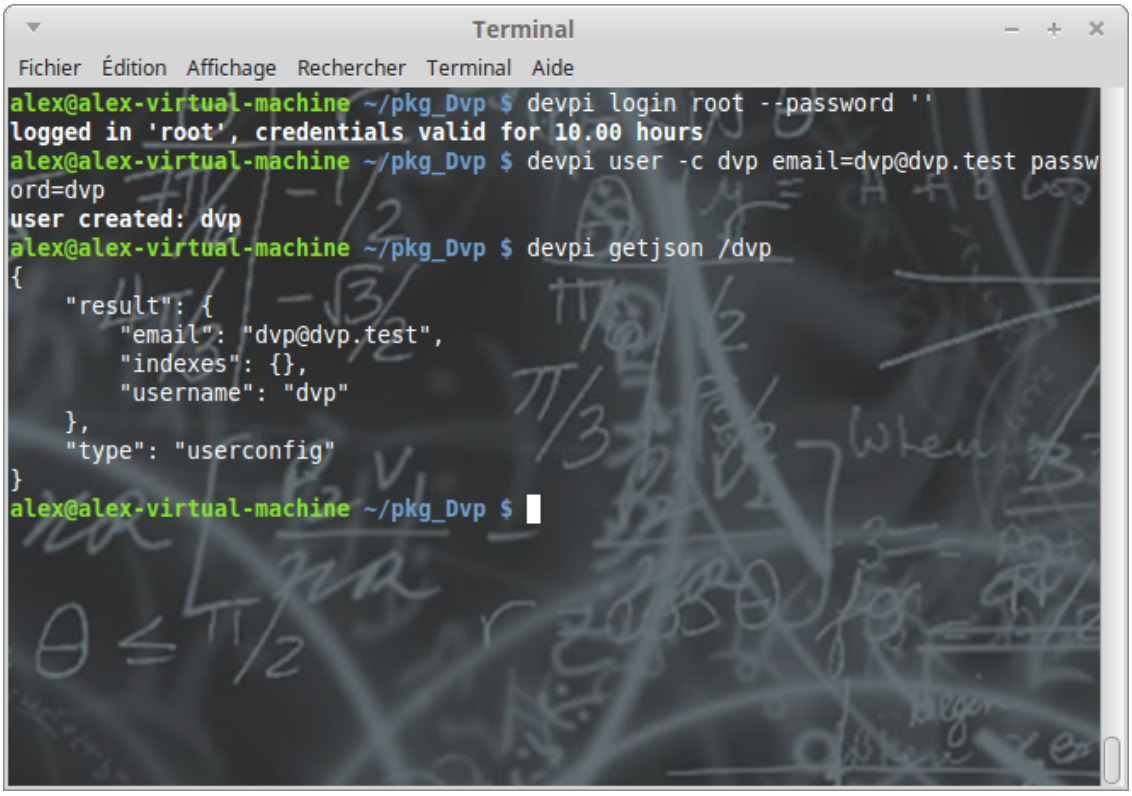
VII-E-2 - Ajout/modification/suppression d'un utilisateur

Un utilisateur sera identifié par son pseudonyme, son e-mail, et son mot de passe. L'option « -c » est utilisée pour la création d'un nouvel utilisateur :

```
devpi user -c dvp email=dvp@dvp.test password=dvp
```

Une fois l'utilisateur créé, vous pouvez contrôler ses paramètres aisément (hors mot de passe) :

```
devpi getjson /dvp
```



En cas d'erreur, ou de besoin, vous pouvez modifier l'utilisateur via l'option « -m » . Il vous faudra toutefois vous authentifier auparavant, si cela n'est pas déjà fait :

```
devpi login dvp --password dvp
devpi user -m dvp email=dvp@developpez.com
```

Seuls l'e-mail et le mot de passe sont modifiables.

Si l'utilisateur n'a plus de raison d'exister, il suffit de se connecter en root (seul utilisateur ayant les droits adéquats), puis de le supprimer (l'option « -y » est là pour ne pas avoir à confirmer la suppression):

```
devpi user dvp -y --delete
```

Enfin, pour connaître la liste des utilisateurs créés, saisissez la ligne suivante :

```
devpi user -l
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

VII-E-3 - Définir les droits admin pour la gestion des utilisateurs

Par défaut, tout utilisateur possède le droit de créer d'autres d'utilisateurs. Il est cependant possible d'empêcher cela via l'option --restrict-modify :

```
devpi-server --restrict-modify root
```

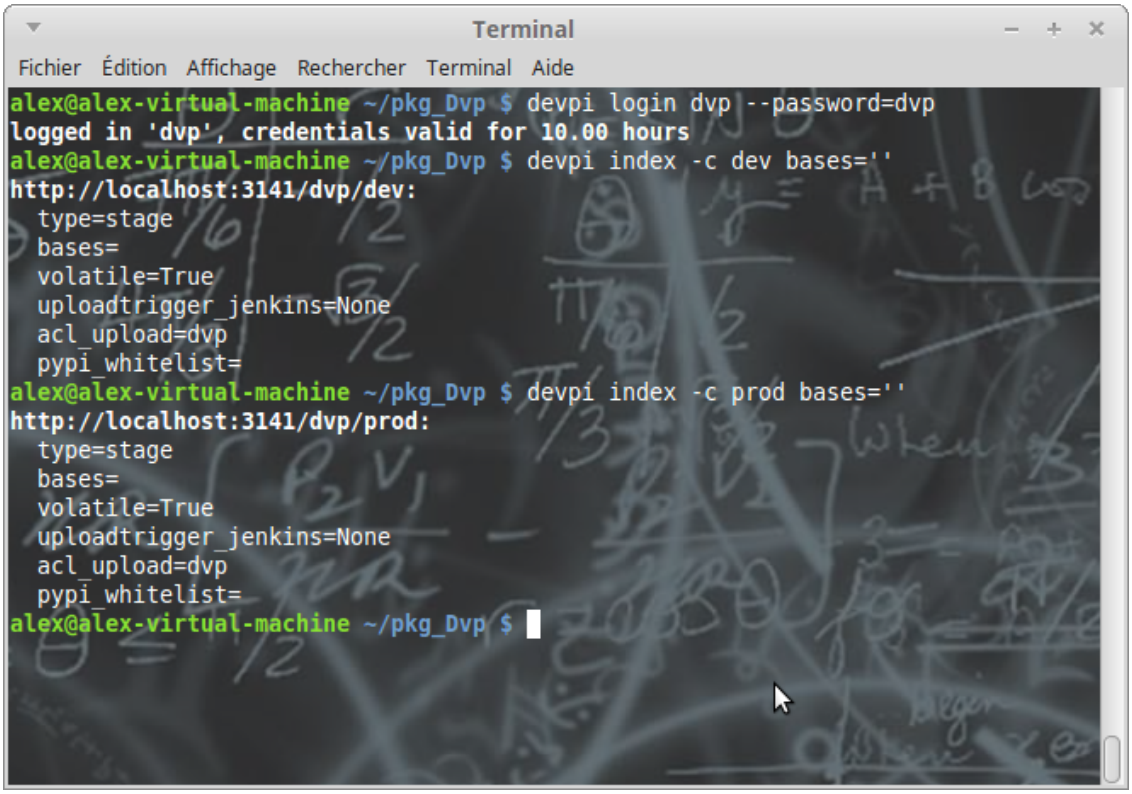
VII-F - Index

Par défaut, devpi crée un index de PyPI (/root/PyPI), et charge en cache, depuis PyPI tout paquet que vous demanderez dans cet index. Cela nécessite donc une connexion Internet.

VII-F-1 - Créer un index vierge

Connectez-vous avec votre user, utilisez la commande adéquate, en spécifiant une base vide :

```
devpi login dvp -password dvp
devpi index -c dev bases=''
```



VII-F-2 - Créer un index héritant d'un autre index

Si toutefois vous désirez non pas un index vierge, mais héritant d'un autre index (bases communes, mais évolutions différentes par exemple), il vous suffit de stipuler le nom de l'index en paramètre, dans « bases » :

```
devpi index -c dev_ihm bases=/dvp/dev
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

VII-F-3 - Associer un index à un utilisateur

Il est tout à fait possible, depuis l'utilisateur root, de définir des index pour les autres utilisateurs.

Pour cela, il suffit de stipuler le chemin de l'index :

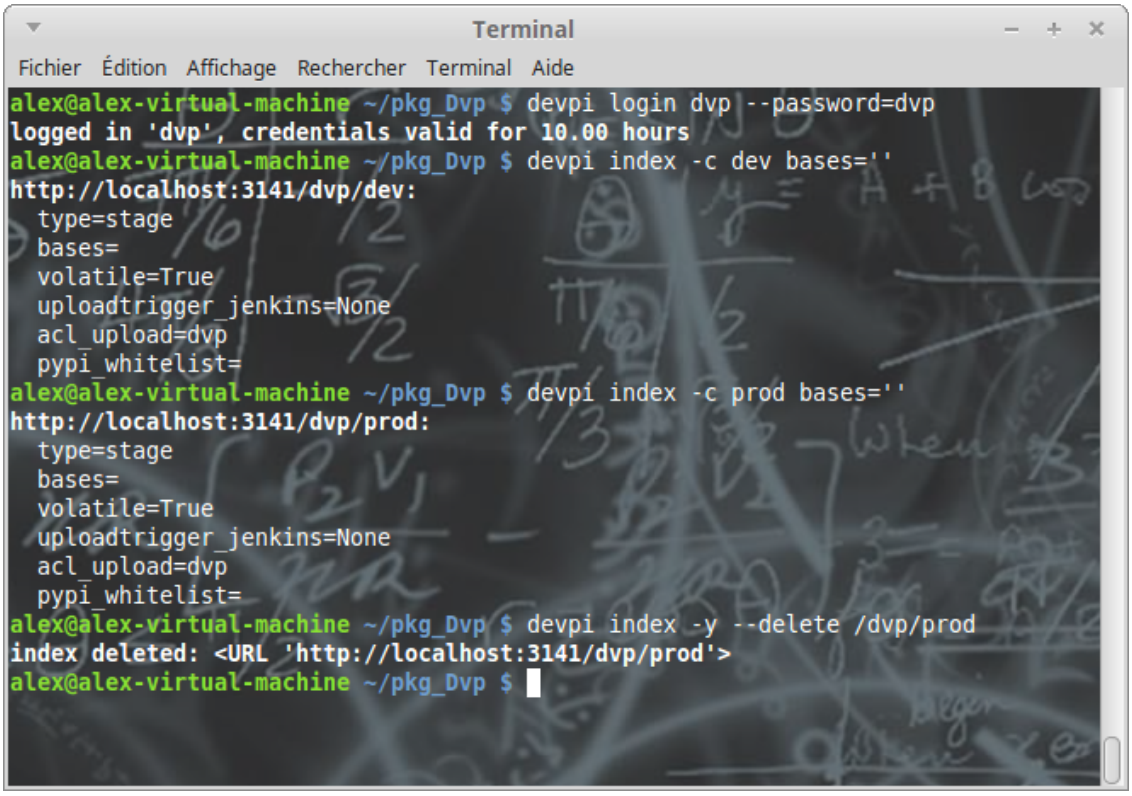
```
devpi index -c /dvp/dev bases=''
```

VII-F-4 - Supprimer un index

Si vous vous êtes trompé à la création d'un index, si un projet est avorté, ou si pour toute autre raison, vous désirez supprimer un index, la commande est la suivante :

```
devpi index -y -delete /dvp/bad_index
```

L'effacement est non réversible.

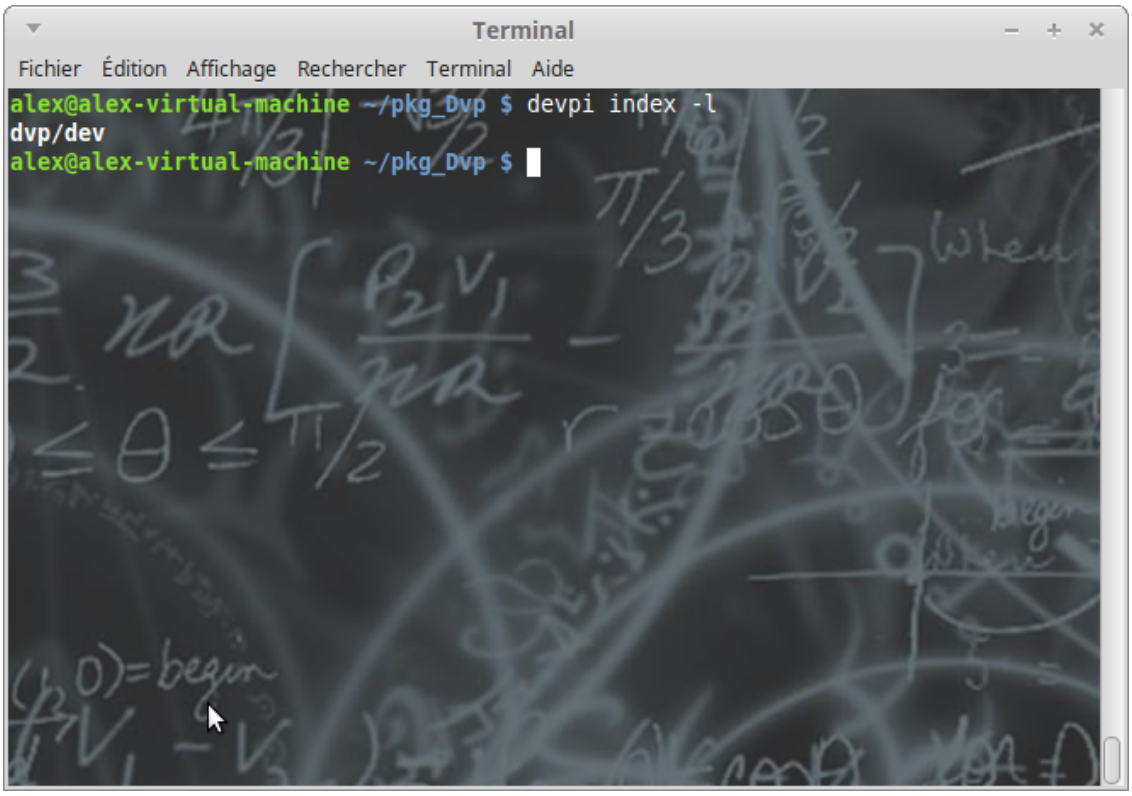


VII-F-5 - Afficher la liste des index disponibles

Pour afficher la liste des index disponibles, il suffit d'utiliser la commande :

```
devpi index -l
```

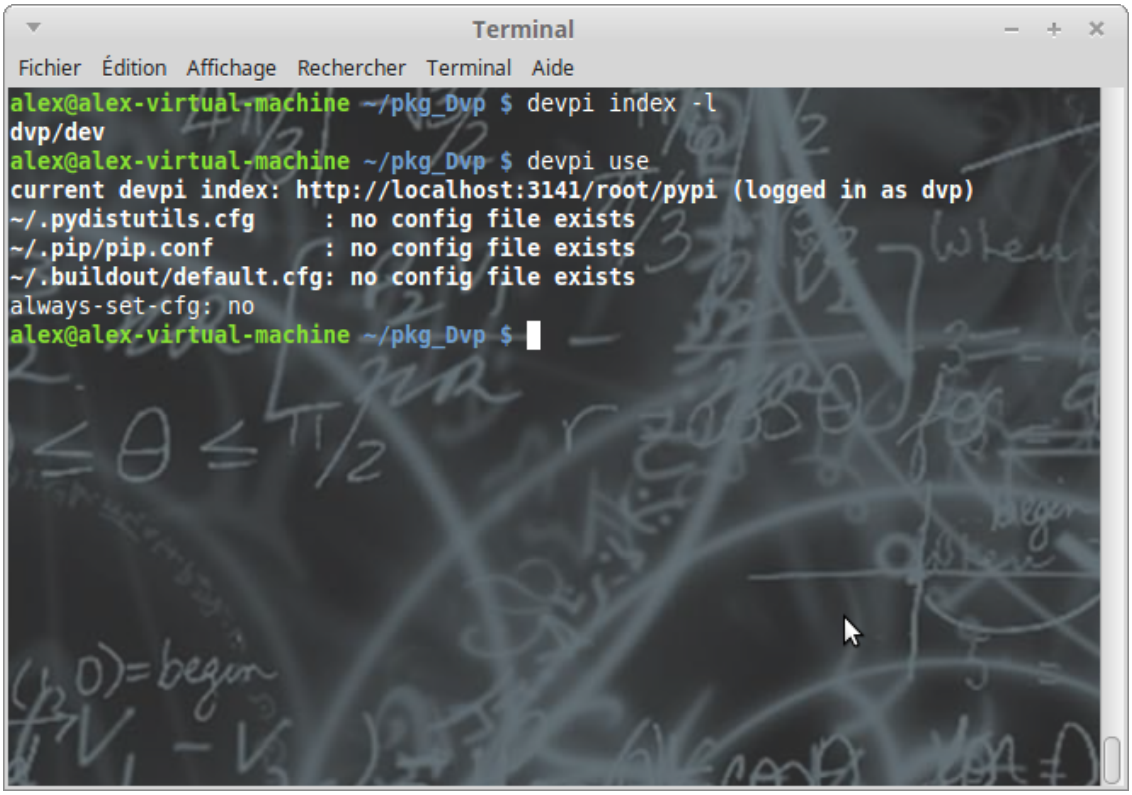

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.



VII-F-6 - Connaître l'index courant

Afin de connaître l'index courant, il suffit de saisir la ligne suivante :

```
devpi use
```



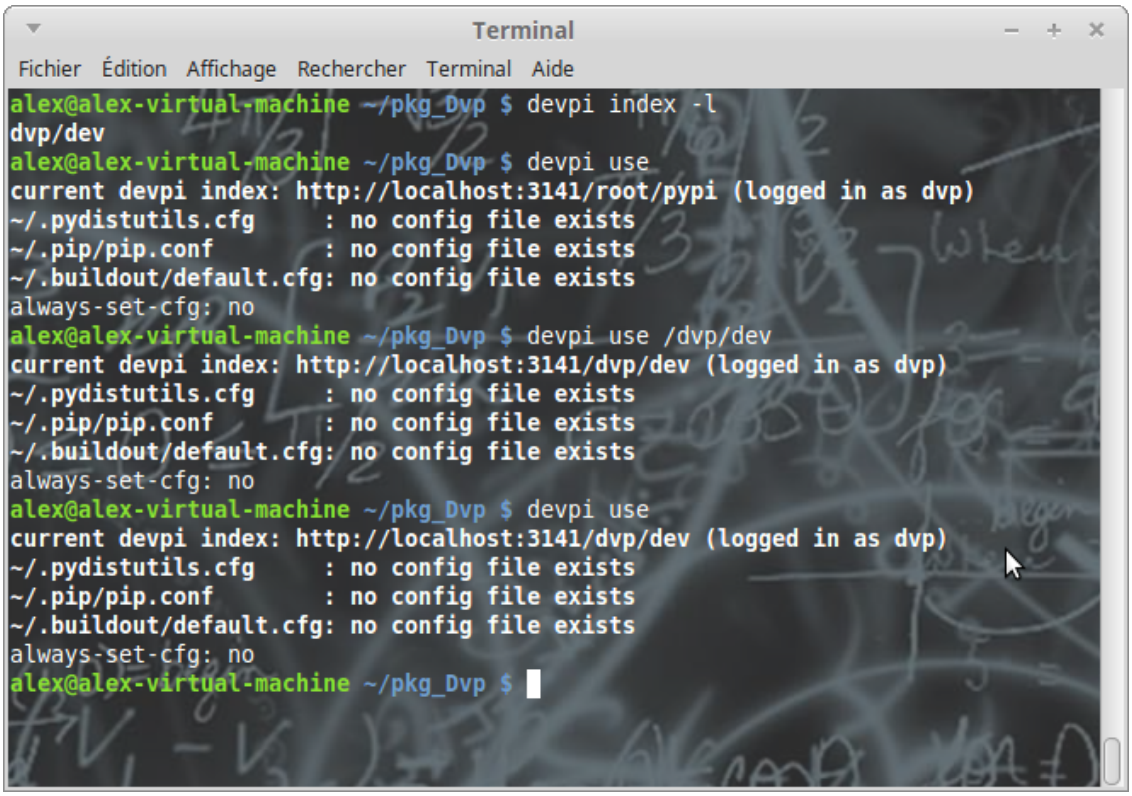
Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

VII-F-7 - Définir l'index courant

Si vous disposez de plusieurs index, il peut être utile de définir l'index désiré, afin qu'il devienne l'index courant. Pour cela, utilisez la commande :

```
devpi use <indexname>
```

avec <indexname> du type /dvp/nom_index



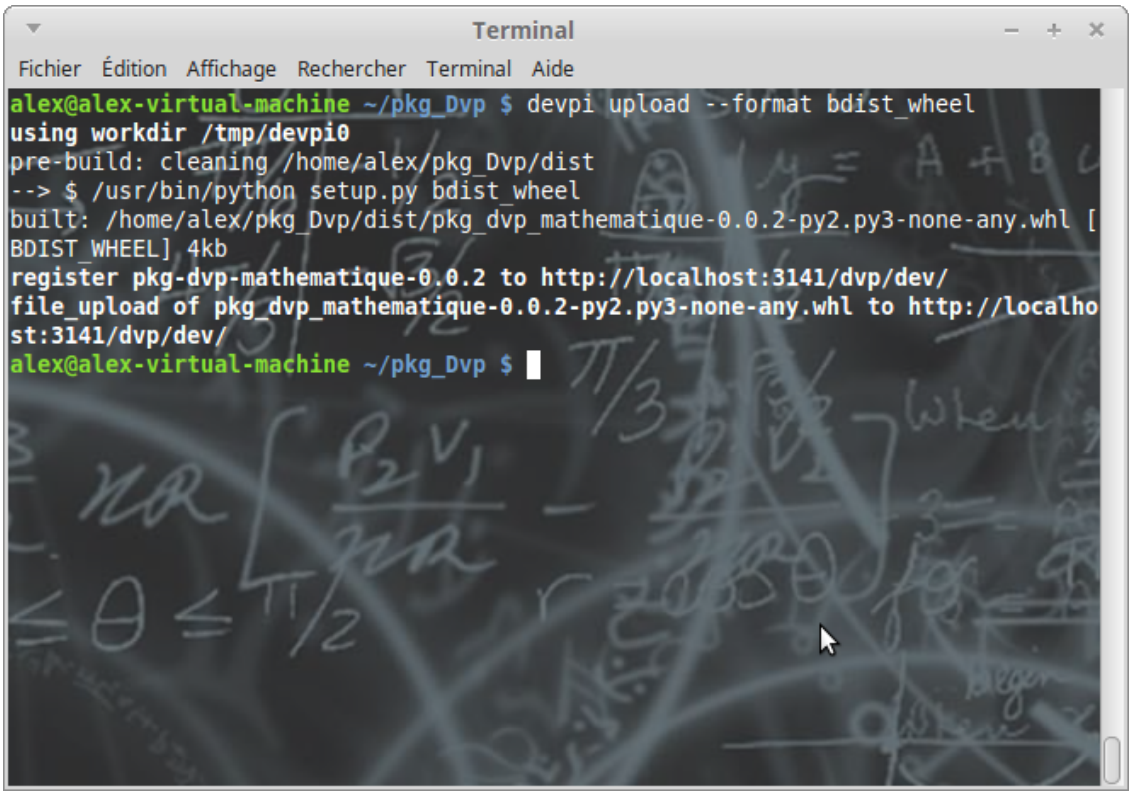
VII-G - Ajout d'un paquet

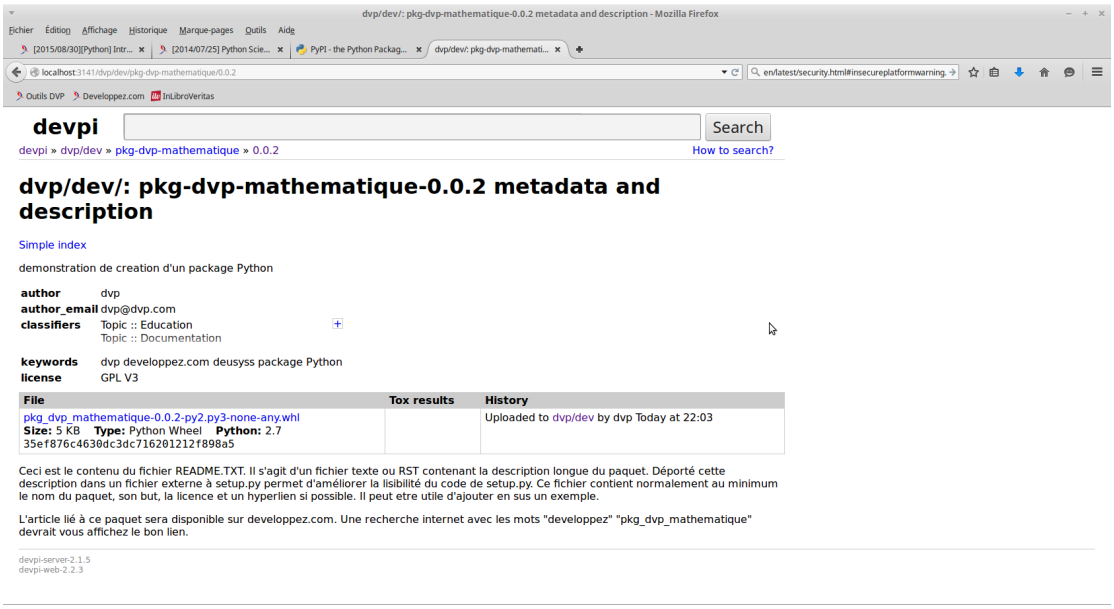
Pour uploader un paquet, placez-vous dans le dossier contenant le setup.py. Assurez-vous d'être dans le bon index, puis saisissez :

```
devpi upload --format bdist_wheel
```

Le paquet va alors être créé et chargé dans l'index. S'il s'agit d'un nouveau projet, une entrée sera créée dans l'index.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.





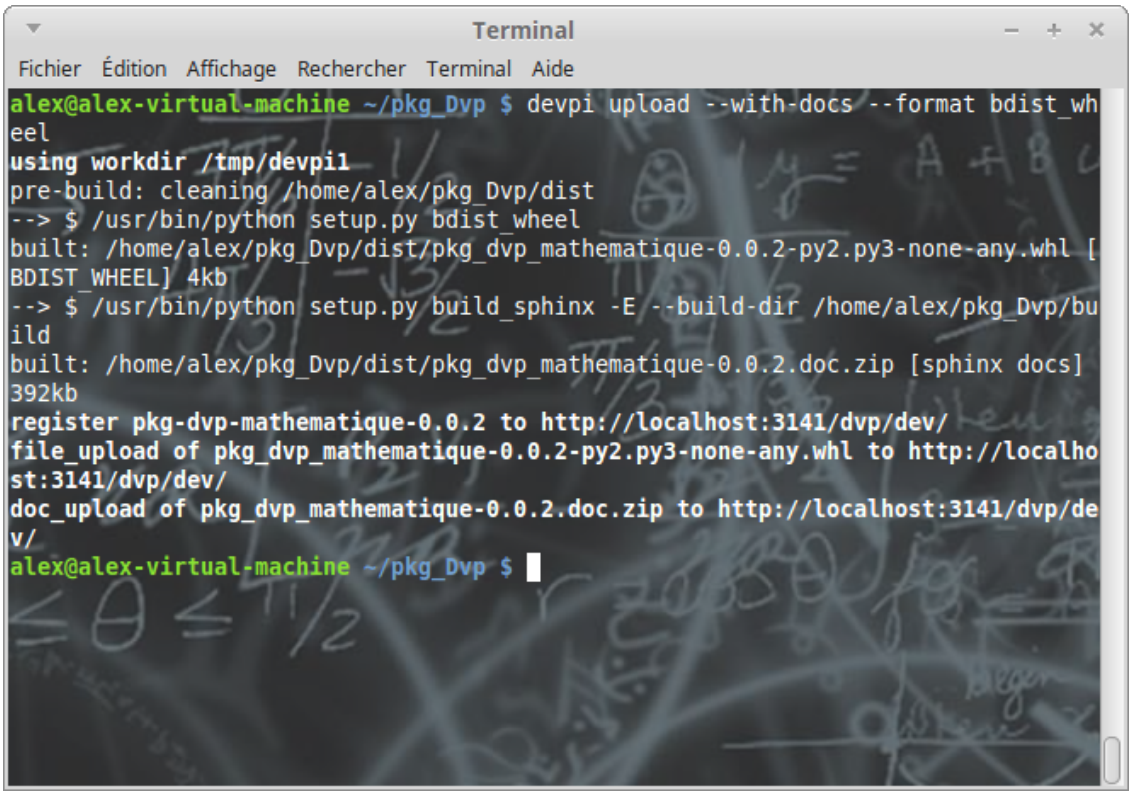
VII-H - Ajout de documentation Sphinx

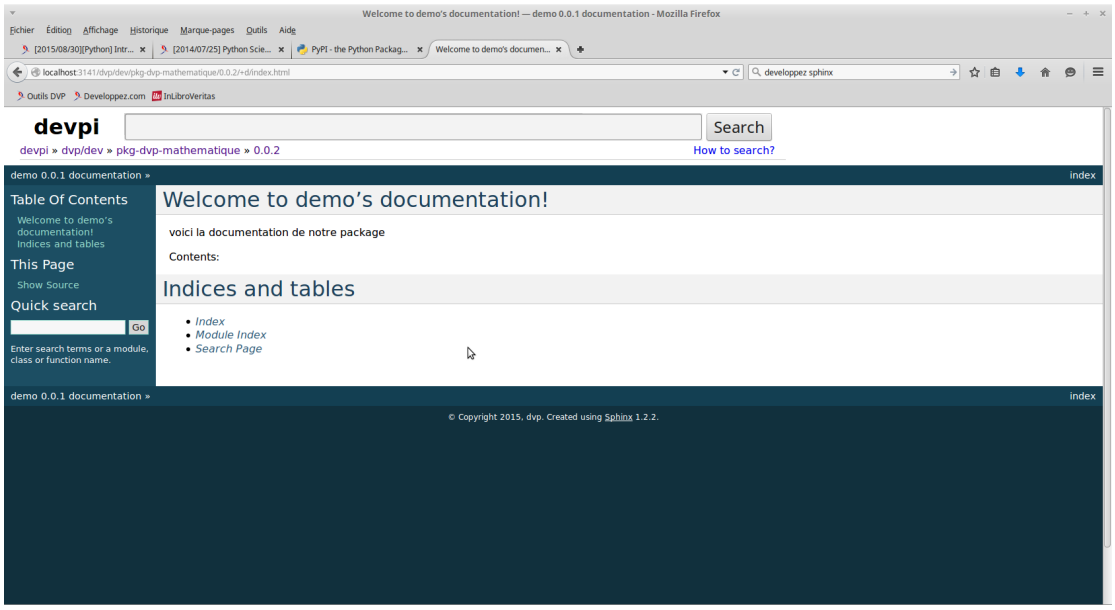
Lors d'un upload, vous avez la possibilité de charger une documentation associée à votre package. Pour cela, cette documentation doit être écrite au format **Sphinx**. Cette documentation doit être placée dans un dossier « doc » situé au niveau du setup.py. Il suffira de préciser qu'il faut construire la doc et l'uploader, lors de l'ajout du package :

```
devpi upload --with-docs --format bdist_wheel
```

Bien entendu, vous pourrez joindre dans cette documentation tout fichier complémentaire que vous voudriez joindre.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/





VII-I - Mise à jour

Pour effectuer une mise à jour, il suffit de réexécuter la même commande que pour un ajout.

VII-J - Récupération d'un paquet

Dans notre cas précis, nous allons utiliser l'option « -i » de PyPI, nous permettant de lui indiquer l'adresse d'un PyPI différent du PyPI officiel.

```
pip install -i http://localhost:3141/root/PyPI/ <nom_du_paquet>
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/

Bien entendu, si vous désirez une version précise du paquet, cela se passe via un format requirement.

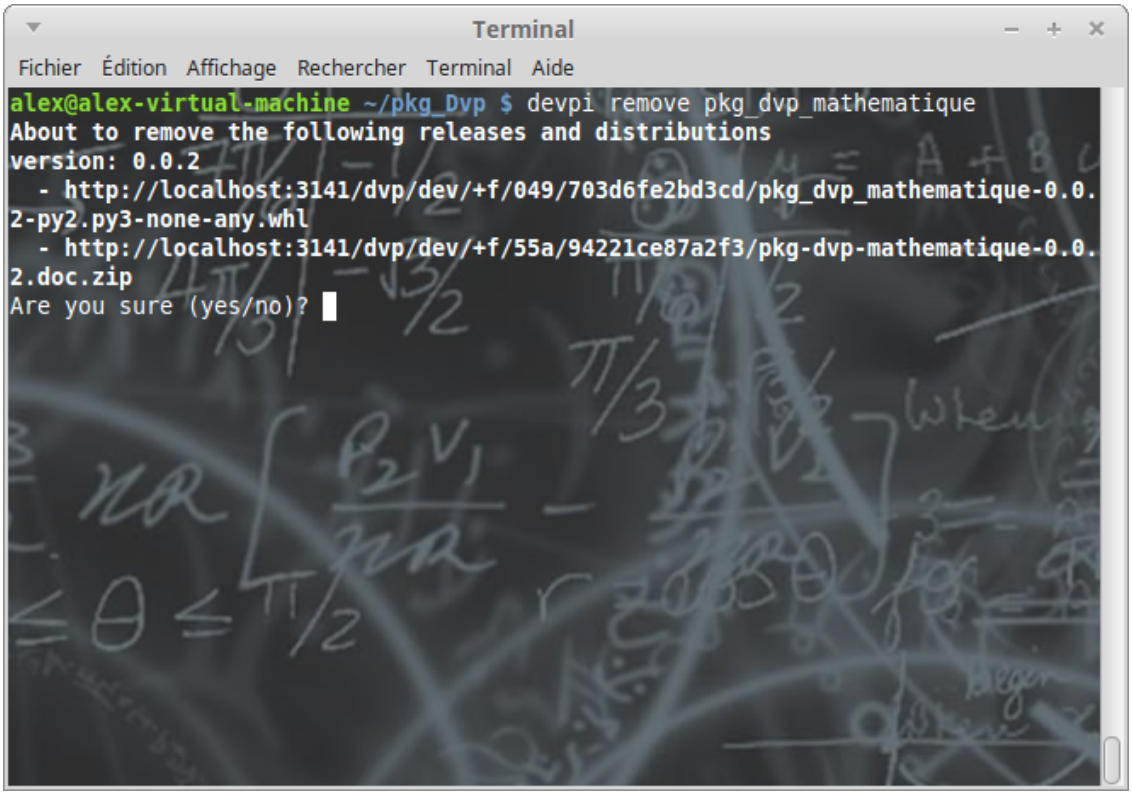
Pour installer un ensemble de paquets listés dans un fichier de requirement :

```
pip install -i http://localhost:3141/root/PyPI/ -r requirement.txt
```

VII-K - suppression d'un paquet

La suppression d'un package et des fichiers associés passe par la commande remove. Il ne faudra pas préalablement oublier de définir l'index courant adéquat.

```
devpi remove <nom_du_package>
```



Notez que vous pouvez fort bien ne retirer qu'une version donnée via le format requirement :

```
devpi remove <nom_du_package>==2.0
```

Si vous supprimez l'intégralité des versions d'un package donné, son entrée dans l'index est automatiquement supprimée.

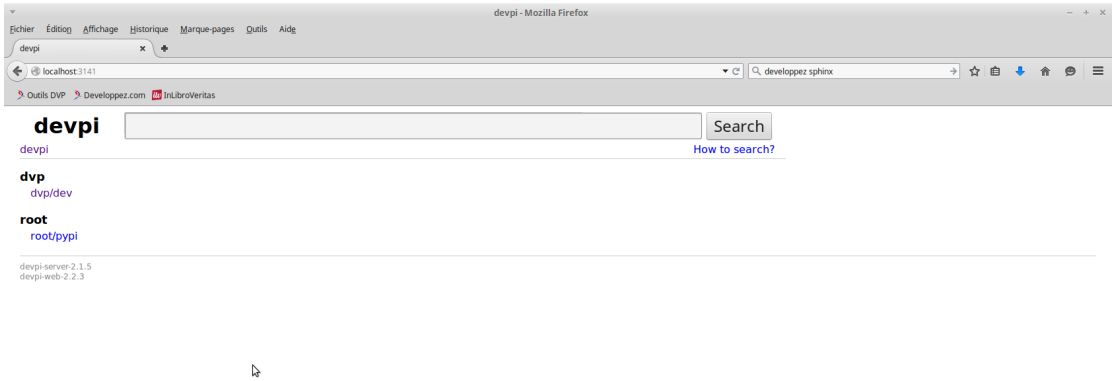
VII-L - Navigation dans devpi

Maintenant que nous avons vu comment alimenter notre serveur personnel de packages Python, nous allons voir comment naviguer à l'intérieur de celui-ci.

VII-L-1 - Page d'accueil

Rendez-vous sur http://localhost:3141

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



VII-L-2 - Afficher un index sous forme de tableau

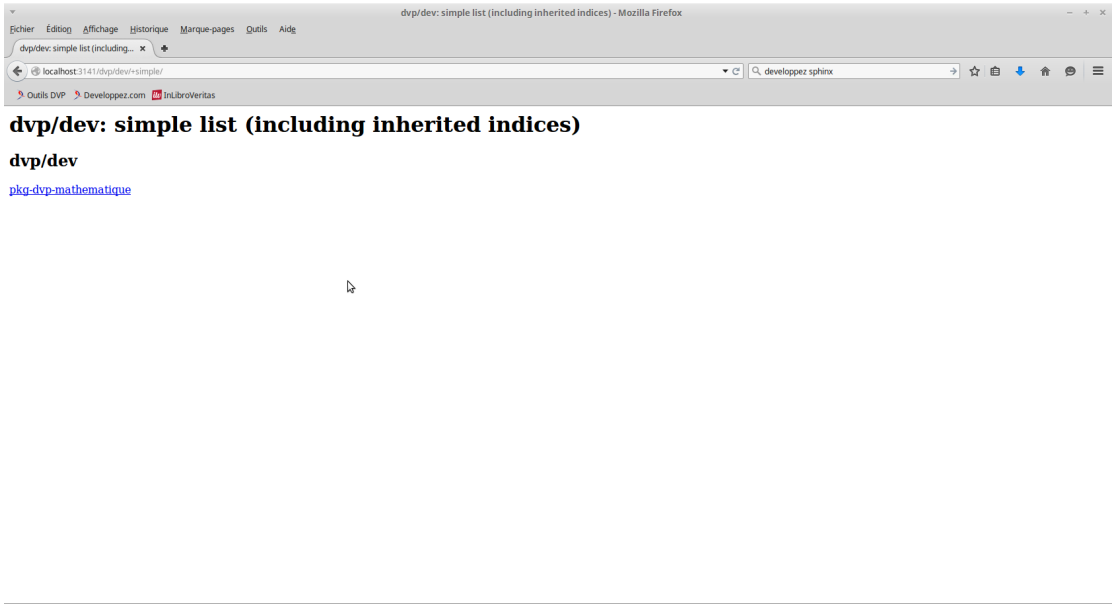
Puis cliquez sur le lien « /dvp/dev » afin d'ouvrir la page d'accueil de l'index



VII-L-3 - Afficher un index sous forme de listing

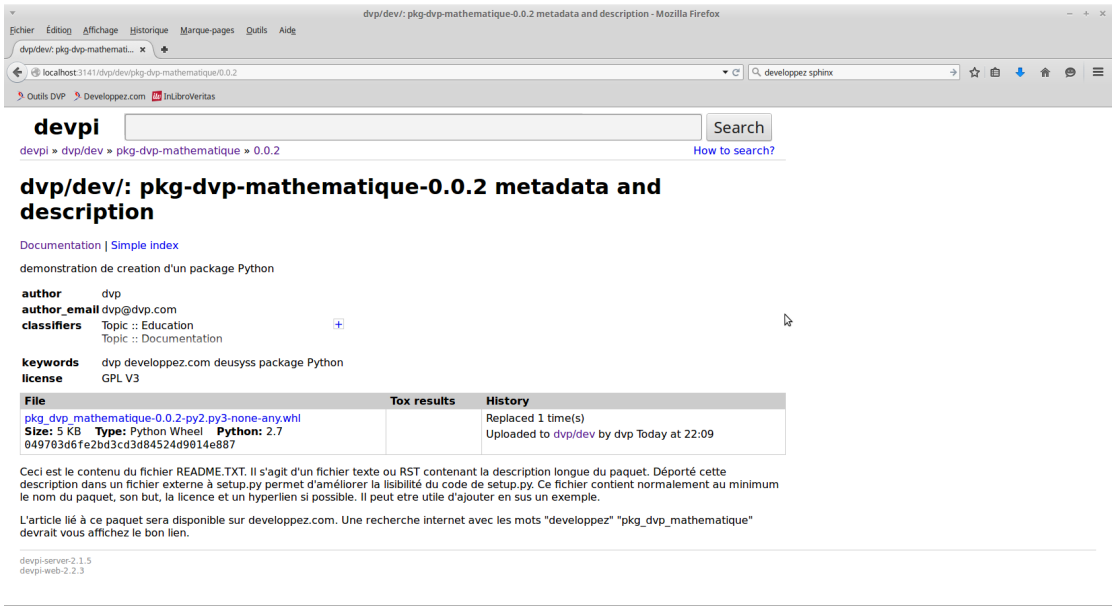
Depuis cette page, vous pouvez changer de type d'affichage, en cliquant sur le lien « simple index »

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.



VII-L-4 - Page d'accueil d'un package donné

Cliquez sur le nom du package dans le listing, ou dans la colonne « info page », depuis l'index au format tableau.



Cette page correspond à la dernière version.

VII-L-5 - Afficher l'historique d'un package

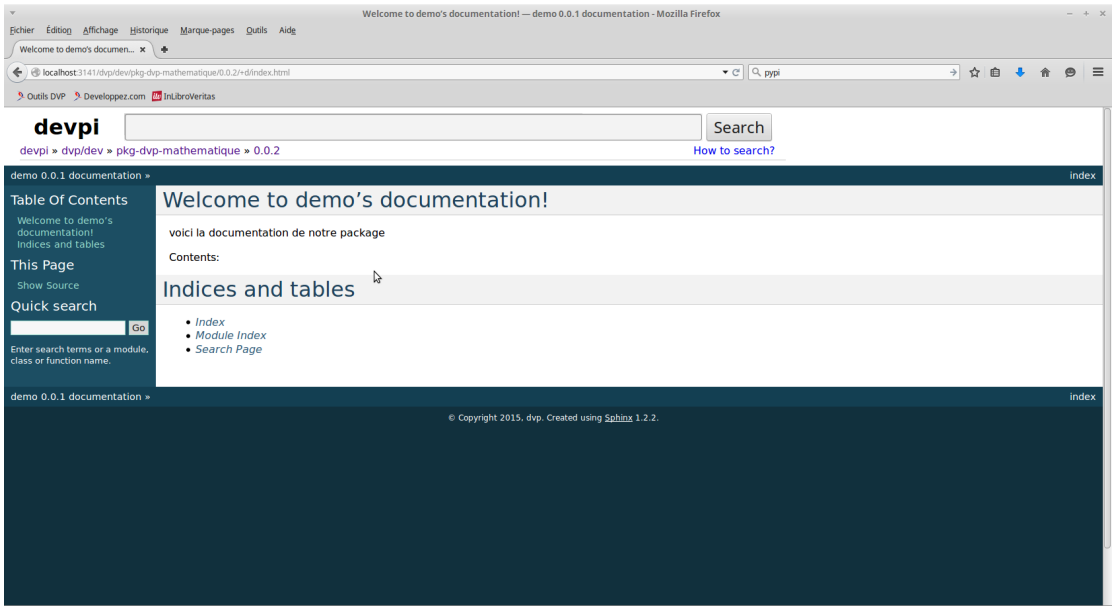
Depuis la page précédente, cliquez sur le nom du package, juste sous le champ de recherche, afin d'afficher l'ensemble des révisions disponibles.

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/



VII-L-6 - Affichage de la documentation

Depuis la page d'accueil d'une version donnée d'un package, vous pouvez cliquer sur le lien « documentation » afin d'afficher la documentation liée à cette version du package.



VII-M - Annexe : Les commandes de référence devpi

Voici le détail des principales commandes fournies avec devpi. En fond jaune, les options génériques, entendez par là qu'elles fonctionnent quelle que soit la commande.

VII-M-1 - getjson

Cette commande permet d'afficher le contenu du json d'un utilisateur.

```
devpi getjson [option] path
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Option/paramètre	Description
path	Chemin du json désiré (ex. : /user ou encore /user/index)
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-2 - Index

Cette commande permet de créer, effacer et manipuler les index. Les index sont toujours créés sous l'utilisateur avec lequel nous sommes connectés.

```
devpi index [option] [indexname] [keyvalue]
```

Option/paramètre	Description
indexname	Permet de stipuler le nom de l'index ou user/index
keyvalues	Permet de stipuler si l'index doit être volatil (volatile=True) ou non (volatile=False).
-c / --create	Permet de créer un index
--delete	Permet de supprimer un index
-l / --list	Permet d'afficher la liste des index liés à l'utilisateur connecté
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-3 - Install

Cette commande permet d'installer des paquets de devpi sans passer par la commande Pip (qui est tout de même appelée en arrière-plan).

```
devpi install [option] pkg
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Option/paramètre	Description
pkg	Paquet à installer
-l	Permet d'afficher la liste des paquets déjà installés
-e ARG	Permet d'installer un projet en mode éditable
--venv DIR	Permet de stipuler le dossier d'une VENV à l'intérieur de laquelle nous désirons installer le paquet
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-4 - List

Cette commande permet d'afficher la liste des projets ou la liste des versions disponibles d'un projet donné de l'index courant. Sans argument **spec** spécifié, la commande affiche le nom de tous les projets disponibles.

```
devpi list [option] [spec]
```

Option/paramètre	Description
spec	Nom d'un projet dont on désire connaître les différentes versions disponibles
-f / --failure	Affiche les logs des setup/failures
--all	Affiche l'ensemble des versions au lieu de juste afficher la plus récente
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-5 - Login

Cette commande permet de se connecter avec un utilisateur spécifique au devpi.

```
devpi login [option] username
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Option/paramètre	Description
username	Nom de l'utilisateur avec lequel on désire se connecter
--password PASSWORD	Permet de passer le mot de passe de l'utilisateur
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-6 - Logoff

Cette commande permet de se déconnecter de devpi.

```
devpi logoff [option]
```

Option/paramètre	Description
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-7 - Push

Cette commande permet de pousser une nouvelle version ou des fichiers liés vers un index donné.

```
devpi push [option] pkg
```

Option/paramètre	Description
pkg	Nom du paquet au format requirement (nom==version)
--PyPIrc PATH	Permet de stipuler le chemin vers le PyPIrc (fichier de paramétrage)
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-8 - Quickstart

Cette commande permet de simplifier toutes les démarches basiques lors d'une mise en place :

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

- démarrage du serveur ;
- création d'un user et connexion ;
- création d'un index de type user/dev et connexion.

```
devpi quickstart [option]
```

Option/paramètre	Description
--user USER	Permet de stipuler le nom de l'utilisateur à créer
--password PASSWORD	Permet d'initialiser le mot de passe du nouvel utilisateur. Il est vide par défaut
--index INDEX	Permet de définir le nom de l'index par défaut pour l'utilisateur
--dry-run	N'effectuer aucune action, juste les afficher
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-9 - Remove

Cette commande permet de retirer un projet complet de l'index courant.

```
devpi remove [option] spec
```

Option/paramètre	Description
spec	Nom du projet au format requirement
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-10 - Test

Cette commande permet de télécharger un paquet et de démarrer les tests TOX configurés dans le fichier tox.ini (qui doit être dans le dossier du paquet)

```
devpi test [option] pkgspec
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Option/paramètre	Description
pkgspec	Nom du paquet au format requirement. Il est possible d'en spécifier plusieurs à suivre, séparés d'un espace
-e ENVNAME	Définit dans tox.ini, nom de l'environnement tox de test à utiliser
-c PATH	Fichier de configuration tox à utiliser
--fallback-ini PATH	Fichier tox.ini de secours à utiliser si besoin
--tox-args TOXARGS	Permet de spécifier des arguments pour le test
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-11 - Upload

Cette commande permet de construire et de charger un paquet sur un index donné du serveur.

```
devpi upload [option]
```

Option/paramètre	Description
--no-vcs	
--formats FORMATS	Permet de stipuler les formats que l'on désire utiliser pour la construction (ex. : bdist_wheel)
--with-docs	Appelle « setup.py build_sphinx » pour construire la documentation Sphinx, puis l'upload
--only-docs	Ne réalise aucune construction autre que la documentation
--from-dir	Permet de stipuler un dossier qui sera parcouru de manière récursive à la recherche d'une archive
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-12 - Use

Cette commande permet d'afficher et de configurer l'index courant et de la venv cible.

```
devpi use [option] url
```


Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

Option/paramètre	Description
url	Permet de stipuler une URL à utiliser. On peut définir un utilisateur et un index en particulier (ex. : https://username:password@example.com/user/index)
--set-cfg	Permet de créer ou de modifier des fichiers de configuration pour Pip/setuptools dans le dossier home
--always-set-cfg YES/NO	Si utilisé avec « yes », alors l'ensemble des commandes « devpi use » utilisera le paramétrage défini avec « --set-cfg »
--venv VENV	Permet de stipuler une venv à utiliser pour les installations
--urls	Permet d'afficher les URL utilisées
-l	Affiche l'ensemble des index disponibles sur le serveur visé
--delete	Supprime la connexion active avec le serveur
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

VII-M-13 - User

Cette commande permet de gérer les utilisateurs du serveur.

```
devpi user [option] username [keyvalues]
```

Option/paramètre	Description
username	Permet de stipuler le nom d'un utilisateur
keyvalues	Permet de stipuler un paramètre donné (e-mail ou password)
-c / --create	Permet de créer un utilisateur
--delete	Permet de supprimer un utilisateur
-m / --modify	Permet de modifier un utilisateur (mot de passe ou e-mail seulement)
-l / --list	Permet d'afficher la liste des utilisateurs existants
-h / --help	Affiche l'aide de la commande
--version	Affiche la version du programme
--debug	Affiche les messages de débogage incluant les requêtes au serveur
-y	Permet de répondre « oui » à toute question qui serait posée
-v / --verbose	Augmente la verbosité de la commande
--client DIR	Dossier où seront stockées les données client

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.

VIII - Conclusion

Comme nous venons de le voir, la fondation Python continue à travailler activement afin de standardiser et optimiser encore et toujours les procédures utilisées par les développeurs.

Le couple PyPI/Pip se révèle d'une puissance indéniable pour le stockage et le déploiement de paquets Python. Et la possibilité, offerte par devpi, d'en disposer aussi bien avec que sans connexion Internet, aussi bien en local qu'en réseau n'est qu'un plus en sa faveur.

De même, disposer de formats standards de paquets est très positif. En effet, ainsi standardisé, le stockage des différents paquets ne s'en trouve qu'amélioré. Surtout que créer un package est une chose relativement aisée.

Cet article, que j'espère complet même si non exhaustif, vous donnera normalement les bases afin de désormais mieux vous organiser, et d'optimiser vos déploiements.

IX - Remerciements

- **f-leb**
- **ClaudeLELOUP**

Le contenu de cet article est rédigé par GALODE Alexandre et est mis à disposition selon les termes de la Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 3.0 non transposé.
Les logos Developpez.com, en-tête, pied de page, css, et look & feel de l'article sont Copyright ® 2013 Developpez.com.
http://deusyss.developpez.com/tutoriels/Python/packaging_pypi/